

# Dédicace

---

*Je voudrais tout d'abord remercier mon Dieu qui m'a aidé à chaque pas pour que je puisse réaliser mes ambitions. Je dédie ce mémoire de fin d'études :*

***A mon très cher père Chokri :** Pour vos immenses sacrifices, votre courage et surtout votre patience et votre compréhension. Grâce à votre bienveillance, à votre encouragement et à votre générosité, j'ai pu terminer mes études dans L'enthousiasme. Je ferai de mon mieux pour rester un sujet de fierté à vos yeux avec l'espoir de ne jamais vous décevoir. Que ce travail puisse être le résultat de vos efforts. Que Dieu le tout puissant soit à vos côtés et vous accorde une meilleure santé.*

***Au symbole de douceur et d'affection ma mère Chiraz :** Pour l'amour dont tu m'as toujours entouré, pour le sacrifice et le dévouement dont tu as toujours fait preuve, pour l'encouragement sans limites que tu ne cesses de manifester. Aucun mot, aucune phrase ne peut exprimer mes sentiments profonds d'amour, de respect et de reconnaissance. Que ce modeste travail soit un début de mes récompenses envers toi.*

***A Toute ma Famille :** Qui a été toujours pour moi la garante d'une existence paisible et d'un avenir radieux.*

***A mes chers amis :** Merci pour l'amitié qui nous a unies et pour tous les merveilleux moments et les souvenirs partagés. ...*

**Aymen**

Avant de rédiger ce rapport, nous tenons à remercier toutes les personnes ayant contribué au bon déroulement de notre stage de projet de fin d'études.

Au terme de ce travail nous tenons à remercier vivement notre encadreuse Mme. **Sondos Zina** pour son encadrement, sa disponibilité et ses précieux conseils qui nous ont prodigué pendant la durée de notre stage de fin d'études au sein de la **banque centrale de tunisie**.

Par la suite, nous tenons également à exprimer notre profond respect et à remercier infiniment mon encadrante à ESPRIT Mme. **Manel Khamassi** pour son aide utile et précieuse, son excellent encadrement et ses encouragements qui nous ont motivés à réaliser ce travail.

On adresse nos sincères remerciements et tous nos respects à tous les membres du jury qui ont bien voulu accepter de juger notre rapport de projet de fin d'études.

---

On tient à remercier aussi tous le personnel de la **banque centrale de tunisie** qui nous a aidés et nous a encouragés à réaliser ce rapport et pour l'ambiance amicale tout le long de ce projet.

Il est indispensable de ne pas rater cette occasion pour exprimer notre gratitude et notre respect à **nos enseignants à ESPRIT** pour leur collaboration et soutien durant nos études.

# Abréviations

---

|                |                                            |
|----------------|--------------------------------------------|
| <b>API</b>     | Application Programming Interface          |
| <b>AHP</b>     | Analytic Hierarchy Process                 |
| <b>AST</b>     | Abstract Syntax Tree                       |
| <b>AUC</b>     | Area Under the Curve                       |
| <b>CSV</b>     | Comma-Separated Values                     |
| <b>DPG</b>     | Digital Public Goods                       |
| <b>DPI</b>     | Digital Public Infrastructure              |
| <b>G2P</b>     | Government-to-Person                       |
| <b>HTTP</b>    | HyperText Transfer Protocol                |
| <b>JSON</b>    | JavaScript Object Notation                 |
| <b>JWT</b>     | JSON Web Token                             |
| <b>LLM</b>     | Large Language Model                       |
| <b>ML</b>      | Machine Learning                           |
| <b>MLflow</b>  | Machine Learning Flow                      |
| <b>NLP</b>     | Natural Language Processing                |
| <b>ORM</b>     | Object Relational Mapping                  |
| <b>PIAF</b>    | Pipeline Intégré d'Analyse de Fraude       |
| <b>PMT</b>     | Proxy Means Test                           |
| <b>REST</b>    | Representational State Transfer            |
| <b>ROC</b>     | Receiver Operating Characteristic          |
| <b>SHAP</b>    | SHapley Additive exPlanations              |
| <b>SMOTE</b>   | Synthetic Minority Over-sampling Technique |
| <b>SQL</b>     | Structured Query Language                  |
| <b>UML</b>     | Unified Modeling Language                  |
| <b>XAI</b>     | Explainable Artificial Intelligence        |
| <b>XGBoost</b> | eXtreme Gradient Boosting                  |

# Table des matières

---

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>Contexte général</b>                                                       | <b>1</b>  |
| <b>1 Contexte du projet</b>                                                   | <b>2</b>  |
| 1.1 Introduction :                                                            | 2         |
| 1.2 Présentation de l'organisme d'accueil : EY Tunisie                        | 2         |
| 1.2.1 AMC EY Tunisie                                                          | 3         |
| 1.2.2 Services proposés par EY Tunisie                                        | 3         |
| 1.2.3 Présentation du département Technology Transformation :                 | 4         |
| 1.3 Présentation du projet                                                    | 5         |
| 1.3.1 Cadre du projet                                                         | 5         |
| 1.3.2 Contexte du projet                                                      | 5         |
| 1.3.3 Problématique                                                           | 5         |
| 1.3.4 Étude de l'existant : la plateforme OpenG2P                             | 5         |
| 1.3.4.1 Présentation d'OpenG2P                                                | 6         |
| 1.3.4.2 Processus actuel dans OpenG2P et les limites face au risque de fraude | 6         |
| 1.3.5 Objectif du projet                                                      | 7         |
| 1.4 Choix méthodologique et conceptuel                                        | 8         |
| 1.4.1 Méthodologie de conception                                              | 8         |
| 1.4.2 Méthodologie adoptée                                                    | 8         |
| 1.4.2.1 Comparaison entre les méthodologies traditionnelles et agiles         | 8         |
| 1.4.2.2 Pilotage du projet                                                    | 8         |
| 1.5 Diagramme de Gantt                                                        | 8         |
| <b>2 État de l'art</b>                                                        | <b>10</b> |
| 2.1 Définitions et concepts clés                                              | 10        |
| 2.1.1 Concepts liés à la fraude et à la protection sociale :                  | 10        |
| 2.1.2 Concepts d'Intelligence Artificielle et Machine Learning                | 10        |
| 2.1.3 Métriques d'Évaluation et Concepts de Validité                          | 11        |
| 2.1.4 Explicabilité et génération d'explications                              | 12        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 2.1.5    | Analyse de graphe et détection de réseaux . . . . .                 | 12        |
| 2.1.6    | Modèles de Machine Learning et Architectures de Détection . . . . . | 13        |
| 2.1.7    | Identification des acteurs . . . . .                                | 14        |
| 2.1.8    | Présentation des travaux existants . . . . .                        | 15        |
| <b>3</b> | <b>Spécifications fonctionnelles et choix technologiques</b>        | <b>18</b> |
| 3.1      | Introduction . . . . .                                              | 18        |
| 3.2      | Identification des besoins fonctionnelles . . . . .                 | 18        |
| 3.3      | Identification des besoins non fonctionnels . . . . .               | 19        |
| 3.4      | Diagramme de cas d'utilisation général . . . . .                    | 20        |
| 3.5      | Diagramme de classes général . . . . .                              | 21        |
| 3.6      | Architecture physique . . . . .                                     | 22        |
| 3.7      | Architecture logique . . . . .                                      | 23        |
| 3.8      | Organisation de flux de données . . . . .                           | 24        |
| 3.9      | Spécifications techniques et environnement logiciel . . . . .       | 25        |
| 3.10     | Stack de développement backend et données . . . . .                 | 25        |
| 3.11     | Stack de développement et DevOps . . . . .                          | 26        |
| 3.11.1   | Dashboards et visualisation des résultats . . . . .                 | 26        |
| 3.11.2   | Collaboration et documentation . . . . .                            | 28        |
| 3.12     | Conclusion . . . . .                                                | 28        |
| <b>4</b> | <b>Réalisation</b>                                                  | <b>29</b> |
| 4.1      | Environnement de développement . . . . .                            | 29        |
| 4.1.1    | Configuration Docker Compose . . . . .                              | 30        |
| 4.2      | Architecture logicielle . . . . .                                   | 33        |
| 4.2.1    | Organisation du projet . . . . .                                    | 33        |
| 4.2.2    | Architecture du moteur intelligent . . . . .                        | 34        |
| 4.2.3    | Composants principaux du moteur . . . . .                           | 35        |
| 4.2.4    | Les interactions entre les composants du systeme . . . . .          | 36        |
| 4.3      | Développement des fonctionnalités . . . . .                         | 37        |
| 4.3.1    | Construction du dataset . . . . .                                   | 37        |
| 4.3.1.1  | Extraction SQL et jointures . . . . .                               | 38        |
| 4.3.1.2  | Prétraitement et scénarios de fraude simulés . . . . .              | 39        |
| 4.3.1.3  | Sélection des sources de données . . . . .                          | 39        |

|           |                                                                        |    |
|-----------|------------------------------------------------------------------------|----|
| 4.3.1.4   | Construction des features et tableau des 25 variables . . . . .        | 40 |
| 4.3.1.5   | Structure du dataset final . . . . .                                   | 41 |
| 4.3.1.6   | Génération du dataset synthétique et injection PostgreSQL . .          | 43 |
| 4.3.1.7   | Un second banc d'essai : le dataset PaySim . . . . .                   | 43 |
| 4.3.2     | Moteur de détection de fraude . . . . .                                | 45 |
| 4.3.3     | Conception du moteur de détection de fraude par Machine Learning . . . | 46 |
| 4.3.3.0.1 | Rééquilibrage des classes avec SMOTE. . . . .                          | 46 |
| 4.3.3.0.2 | Entraînement XGBoost. . . . .                                          | 46 |
| 4.3.3.0.3 | Isolation Forest pour la détection d'anomalies. . . . .                | 46 |
| 4.3.4     | Analyse de réseau . . . . .                                            | 47 |
| 4.3.5     | Fusion des scores . . . . .                                            | 47 |
| 4.3.6     | Explicabilité SHAP . . . . .                                           | 47 |
| 4.3.6.0.1 | SHAP . . . . .                                                         | 47 |
| 4.3.7     | Explication LLM . . . . .                                              | 48 |
| 4.3.7.0.1 | LLM Mistral . . . . .                                                  | 48 |
| 4.4       | Intégration dans OpenG2P . . . . .                                     | 48 |
| 4.4.1     | Exposition via FastAPI . . . . .                                       | 48 |
| 4.4.2     | Synchronisation Odoo . . . . .                                         | 48 |
| 4.4.3     | Dashboard Streamlit . . . . .                                          | 49 |
| 4.4.4     | Déploiement Docker . . . . .                                           | 49 |
| 4.5       | Conclusion partielle . . . . .                                         | 49 |

# Table des figures

---

|      |                                                                                                                        |    |
|------|------------------------------------------------------------------------------------------------------------------------|----|
| 1.1  | Logo d'EY . . . . .                                                                                                    | 2  |
| 1.2  | EY Global . . . . .                                                                                                    | 2  |
| 1.3  | AMC EY Tunisie . . . . .                                                                                               | 3  |
| 1.4  | Les activités de la division « Technology Transformation » . . . . .                                                   | 4  |
| 1.5  | Logo de OpenG2P . . . . .                                                                                              | 6  |
| 1.6  | Processus actuel dans OpenG2P et les zones de vulnérabilité à la fraude . . . . .                                      | 7  |
| 1.7  | Diagramme de Gantt du projet . . . . .                                                                                 | 8  |
| 3.1  | Diagramme de cas d'utilisation général . . . . .                                                                       | 20 |
| 3.2  | Diagramme de classe général . . . . .                                                                                  | 21 |
| 3.3  | Architecture physique du moteur intelligent de détection de fraude . . . . .                                           | 22 |
| 3.4  | Architecture logique du moteur intelligent de détection de fraude . . . . .                                            | 23 |
| 3.5  | Diagramme de flux de données du moteur intelligent de détection de fraude . . . . .                                    | 24 |
| 4.1  | Langages et outils utilisés pour le développement du moteur intelligent de détection de fraude . . . . .               | 29 |
| 4.2  | Extrait de la configuration Docker Compose pour le moteur intelligent et la plateforme OpenG2P . . . . .               | 30 |
| 4.3  | Extrait de la configuration Docker Compose pour le moteur intelligent et la plateforme OpenG2P . . . . .               | 31 |
| 4.4  | Extrait des conteneurs Docker Compose pour le moteur intelligent et la plateforme OpenG2P . . . . .                    | 32 |
| 4.5  | Démarrage de l'ensemble du système via Docker Compose . . . . .                                                        | 33 |
| 4.6  | Vision globale sur la structure du code source . . . . .                                                               | 34 |
| 4.7  | Structure du code source du moteur intelligent de détection de fraude . . . . .                                        | 34 |
| 4.8  | La structure des composants principaux du moteur intelligent de détection de fraude . . . . .                          | 35 |
| 4.9  | Processus de scoring d'un bénéficiaire à travers les composants du moteur intelligent de détection de fraude . . . . . | 36 |
| 4.10 | Processus de construction du dataset de détection de fraude à partir des données OpenG2P . . . . .                     | 37 |

|      |                                                                                                                                                           |    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.11 | Complexité du schéma OpenG2P (587 tables) et colonnes disponibles dans les 7 tables retenues pour l'extraction . . . . .                                  | 38 |
| 4.12 | Scénarios de fraude simulés . . . . .                                                                                                                     | 39 |
| 4.13 | Colonnes et types du dataset OpenG2P généré (36 colonnes, dont les 25 features ML) . . . . .                                                              | 42 |
| 4.14 | Vue d'ensemble du dataset PaySim : distribution des montants par classe, taux de fraude par heure, et comparaison de performance avec le moteur OpenG2P . | 44 |
| 4.15 | Dashboard Streamlit : vue des alertes actives avec niveaux de risque . . . . .                                                                            | 49 |
| 4.16 | Graphique SHAP beeswarm : importance des features sur l'ensemble des prédictions . . . . .                                                                | 49 |

# Table des tableaux

---

|     |                                                                                                             |    |
|-----|-------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Services professionnels d'EY Tunisie . . . . .                                                              | 4  |
| 1.2 | Méthodologies Traditionnelles vs Agiles . . . . .                                                           | 8  |
| 2.1 | Acteurs internes du système . . . . .                                                                       | 14 |
| 2.2 | Acteurs externes et systèmes . . . . .                                                                      | 14 |
| 2.3 | Comparaison des méthodes de détection de fraude : avantages et limites . . . . .                            | 15 |
| 2.4 | Solutions commerciales de détection de fraude sur le marché . . . . .                                       | 16 |
| 2.5 | Limites des travaux existants face à la philosophie DPI / DPG . . . . .                                     | 17 |
| 3.1 | Spécification des besoins fonctionnels du moteur intelligent de détection de fraude<br>par acteur . . . . . | 18 |
| 3.2 | Technologies de base du système de détection de fraude . . . . .                                            | 25 |
| 3.3 | Technologies d'explicabilité et de gestion des modèles . . . . .                                            | 26 |
| 3.4 | Outils de développement et d'infrastructure . . . . .                                                       | 26 |
| 3.5 | Dashboards et plateformes de visualisation . . . . .                                                        | 27 |
| 3.6 | Outils de collaboration et de documentation . . . . .                                                       | 28 |
| 4.1 | Conteneurs Docker du système . . . . .                                                                      | 30 |
| 4.2 | Principales sources de données exploitées pour la construction du dataset . . . . .                         | 40 |
| 4.3 | Les 25 features extraites pour le modèle ML . . . . .                                                       | 41 |
| 4.4 | Comparaison des modèles supervisés . . . . .                                                                | 46 |

# Introduction Générale

---

Dans un monde où **1,3 milliard de personnes** vivent encore sous le seuil de pauvreté [1], la protection sociale n'est plus un luxe, c'est un impératif de survie. Chaque jour, des millions de familles dans les pays émergents et en développement dépendent d'un transfert monétaire pour nourrir leurs enfants, payer une consultation médicale ou maintenir un enfant à l'école. Ces programmes de transferts monétaires directs, désignés sous le terme **Government-to-Person (G2P)**, constituent aujourd'hui le mécanisme le plus efficace pour réduire la pauvreté et les inégalités dans les pays les plus vulnérables.

Pour rendre ces programmes possibles à grande échelle, les gouvernements se tournent vers un concept transformateur : **les Infrastructures Publiques Numériques (DPI)**.

**Les DPI (Digital Public Infrastructure)** désignent l'ensemble des systèmes numériques partagés, tels que les systèmes d'identification biométrique, les registres sociaux centralisés ou les plateformes de paiement, qui constituent le socle technologique nécessaire à la fourniture de services publics modernes, transparents et inclusifs.

Afin de développer et d'opérer efficacement ces infrastructures numériques, les gouvernements peuvent s'appuyer sur les **Biens Publics Numériques (Digital Public Goods)**, des solutions logicielles open source conçues pour être réutilisées, adaptées et déployées à grande échelle dans différents contextes nationaux.

**OpenG2P** est une plateforme open source conçue pour digitaliser la gestion des programmes de protection sociale. Déployée dans plusieurs pays tels que le Kenya, le Malawi et la Sierra Leone, OpenG2P constitue une solution robuste pour les institutions engagées dans la modernisation de leurs infrastructures numériques et l'amélioration de la gouvernance des aides sociales sans dépendance à un éditeur privé.

Cependant, les programmes G2P font face à un adversaire silencieux : **la fraude**.

Les tactiques sont multiples : multi-inscription sous des identités différentes, ménages fictifs etc. Selon la Banque mondiale, **jusqu'à 27 % du budget destiné à la protection sociale** est perdu à cause de ces fuites [3].

Le problème est aggravé par le fait que les contrôles actuels interviennent trop tard. Les audits manuels sont lents, coûteux, et ne couvrent qu'un échantillon limité. Les solutions commerciales, quant à elles, sont coûteuses et incompatibles avec la philosophie open-source des DPG.

**C'est dans ce cadre que notre stage effectué au sein d'EY Tunisia s'inscrit** : concevoir un moteur intelligent de détection de fraude libre, proactif et explicable, intégré à OpenG2P, pour protéger les budgets de protection sociale avant même le versement.

Ce rapport détaille la conception, le développement et l'évaluation de ce projet, en mettant l'accent sur la rigueur scientifique, l'adaptabilité à différents contextes nationaux et la capacité à protéger efficacement les budgets d'aide sociale.

# Chapitre 1 Contexte du projet

---

## 1.1 Introduction :

Ce chapitre expose le contexte général des travaux menés, nous allons présenter le contexte global du projet, en débutant par l'introduction de l'organisme d'accueil qui est « EY Tunisie », décrire ses activités et ses différentes équipes. Par la suite, nous procédons à la description de notre problématique qui a mené au développement de ce projet. Enfin, nous expliquons la méthodologie choisie pour accomplir ce travail.

## 1.2 Présentation de l'organisme d'accueil : EY Tunisie



FIGURE 1.1 – Logo d'EY

Fondé en 1849, Ernst and Young est un cabinet d'audit financier et de conseil multinational implanté dans plus de 152 pays organisés en trois régions : Amérique ; Europe, Moyen Orient, Inde et Afrique (EMEIA) ; Asie-Pacifique. EY fait partie des « Big Four », soit les quatre plus grands groupes d'audit à l'échelle mondiale. EY est une organisation de services professionnels approfondie avec plus de 700 bureaux et 350 000 associés et collaborateurs dans le monde entier.

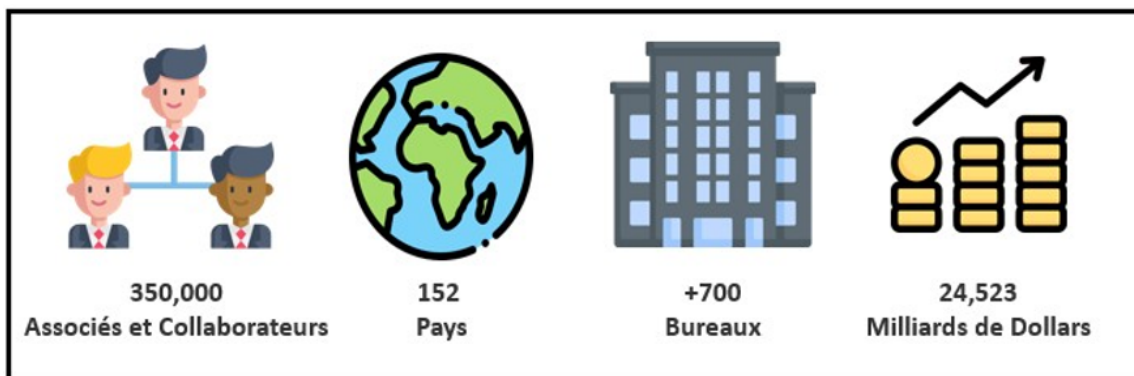


FIGURE 1.2 – EY Global

EY est une entreprise de premier plan à l'échelle mondiale évoluant dans le domaine d'assurance, de fiscalité, de transaction, et de conseil. EY bénéficie d'une équipe mondiale en expansion constante.

### 1.2.1 AMC EY Tunisie

Établie sur le territoire depuis 1987, AMC EY Tunisie se positionne comme l'une des pionnières dans le domaine des services professionnels du pays. Elle fait partie de la région EMEIA et compte plus de 1000 employés avec plus de 900 consultants en service de conseil. Grâce à plus de 620 missions réalisées annuellement, l'entreprise a atteint un chiffre d'affaires de 50 Millions de dinars tunisiens en 2022 avec plus de 50% réalisées auprès de clients internationaux.



FIGURE 1.3 – AMC EY Tunisie

EY Tunisie intervient dans différents secteurs économiques en Tunisie, notamment dans le secteur des technologies de l'information et de la communication, méritant la confiance tant du secteur public local (ministères, agences de développement local) que du secteur privé grâce à la proposition de solutions adaptées et pertinentes.

### 1.2.2 Services proposés par EY Tunisie

EY Tunisie structure son offre autour de quatre grands métiers complémentaires qui permettent à l'entreprise de répondre aux besoins variés de ses clients résumés dans le tableau suivant :

| Service                           | Description                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Conseil</b>                    | EY aide ses clients à créer de la valeur à long terme pour toutes les parties prenantes. En s'appuyant sur les données et la technologie, ses services et ses solutions orientent les clients et les aident à prendre des décisions sur le plan stratégique et opérationnel afin de parvenir favorablement à leurs attentes.                                        |
| <b>Audit</b>                      | EY fournit des renseignements et cherche à offrir aux clients (investisseurs) la meilleure gestion et l'adéquate structuration de ses rapports et de ses opérations dans le but d'améliorer voire de valoriser leurs tâches et d'accroître leurs opportunités envers les autres entreprises.                                                                        |
| <b>Fiscalité et droit (Tax)</b>   | EY possède une expertise en matière d'impôt sur les sociétés, d'impôt international, d'impôt sur les transactions et de ressources humaines, de conformité et de déclaration, et d'impôt statutaire.                                                                                                                                                                |
| <b>Stratégies et Transactions</b> | EY aide ses clients à surmonter la crise et à stabiliser leurs activités à court terme. EY aide également les entreprises à surmonter les ralentissements et à se préparer à la reprise, ou à regarder au-delà pour permettre la transformation par le biais de la stratégie d'entreprise, des transactions, des fusions et acquisitions et des désinvestissements. |

TABLEAU 1.1 – Services professionnels d'EY Tunisie

### 1.2.3 Présentation du département Technology Transformation :

Le département « **Technology Transformation** » qui fait partie du service line Conseil occupe une position centrale dans la transformation d'entreprise et propose des possibilités d'optimisation des rendements, de supervision des risques et de promotion de l'innovation des entreprises.

En s'appuyant sur une gamme diversifiée de projets technologiques, la division « **Technology Transformation** » offre à ses clients la possibilité d'améliorer leurs pratiques et d'acquérir un avantage compétitif. L'équipe offre cinq (5) principales activités pour cinq (5) secteurs intersectoriels, comme l'illustre la figure ci-dessous.

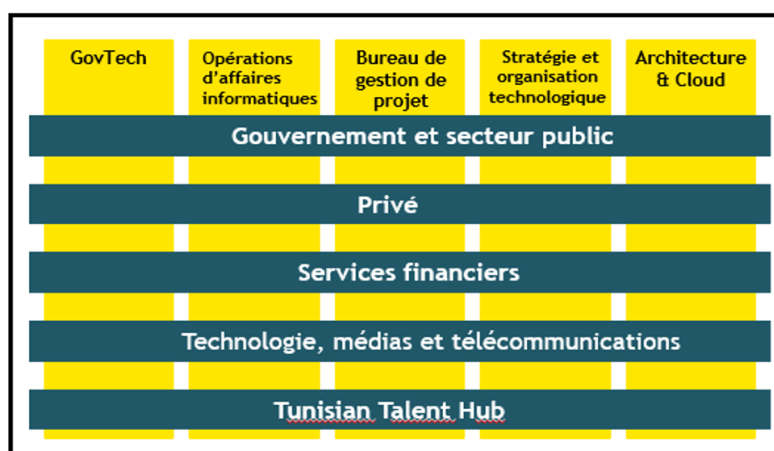


FIGURE 1.4 – Les activités de la division « Technology Transformation »

### 1.3 Présentation du projet

#### 1.3.1 Cadre du projet

Notre projet s'inscrit dans le cadre de la préparation du stage de fin d'études pour l'obtention du Diplôme National d'Ingénieur en Informatique. Notre mission consiste à concevoir et développer un moteur intelligent de détection des risques et fraudes intégré à la plateforme de protection sociale OpenG2P, réalisé au sein du cabinet EY Tunisia.

#### 1.3.2 Contexte du projet

Le projet s'inscrit dans l'axe de modernisation des services publics, où EY accompagne les organismes gouvernementaux dans l'adoption de solutions technologiques modernes. Notre mission porte sur la plateforme OpenG2P, un logiciel libre utilisé par plusieurs pays pour gérer leurs programmes d'aide sociale : identifier les bénéficiaires, organiser les versements, et suivre les paiements.

Afin de bien cerner les enjeux de ce projet, nous avons commencé par étudier en profondeur le fonctionnement d'OpenG2P et de son écosystème. Les observations issues de cette étude nous ont permis d'identifier des lacunes critiques en matière de lutte contre la fraude, que nous détaillons dans les sections suivantes.

#### 1.3.3 Problématique

Les programmes de protection sociale gèrent chaque année des flux financiers considérables destinés aux populations vulnérables. Cependant, selon la Banque mondiale, **jusqu'à 27% de ce budget** est perdu à cause de la fraude et des erreurs de ciblage [3]. Les principaux problèmes identifiés sont les suivants :

- **La fraude prend des formes variées et évolutives** : faux bénéficiaires, ménages fictifs, identités partagées, manipulation de paiements.
- **Les contrôles actuels interviennent après le versement** : quand l'argent a déjà quitté les caisses de l'État.
- **Les vérifications manuelles** sont lentes, coûteuses, et ne couvrent qu'un échantillon limité.
- **Les règles de contrôle figées** ne détectent que les fraudes connues et ne s'adaptent pas aux nouvelles tactiques.
- **Les solutions commerciales** existantes sont **coûteuses, opaques, et incompatibles** avec la philosophie open-source d'OpenG2P.

Il devient donc indispensable de disposer d'une solution intelligente, proactive et explicable, capable de détecter la fraude **avant** le versement.

#### 1.3.4 Étude de l'existant : la plateforme OpenG2P

Avant de proposer une solution, il est essentiel de comprendre comment fonctionne la plateforme sur laquelle elle sera intégrée. Nous avons donc mené une étude approfondie d'OpenG2P,

son architecture, ses modules, et ses processus métier, afin d'identifier précisément où se situent les faiblesses face à la fraude.

### 1.3.4.1 Présentation d'OpenG2P



FIGURE 1.5 – Logo de OpenG2P

**OpenG2P** est une plateforme open-source bâtie sur Odoo 17, conçue pour centraliser la gestion des programmes de protection sociale. Elle est déployée dans plusieurs pays (Kenya, Malawi, Sierra Leone) et couvre l'ensemble du cycle de vie d'un programme social, depuis l'enregistrement des bénéficiaires jusqu'au versement des aides. La plateforme s'organise autour de plusieurs modules interconnectés :

- **Registre social** : base de données centralisée des bénéficiaires avec leurs informations personnelles, démographiques et socio-économiques
- **Gestion des programmes** : définition des programmes d'aide, organisation des cycles de versement (trimestriels, mensuels)
- **Droits et paiements** : calcul automatisé des montants dus à chaque bénéficiaire et suivi de chaque versement
- **Données transactionnelles** : registre des coordonnées téléphoniques et bancaires utilisées pour les transferts
- **Groupes et ménages** : gestion des liens familiaux et des regroupements de bénéficiaires

### 1.3.4.2 Processus actuel dans OpenG2P et les limites face au risque de fraude

Le processus de contrôle anti-fraude dans OpenG2P repose actuellement sur un système minimal : quelques vérifications manuelles effectuées par les agents sociaux lors de l'enregistrement des bénéficiaires, complétées par des audits post-versement sur un échantillon limité. Aucun moteur automatisé de détection n'est intégré à la plateforme.

Afin de mieux comprendre les limites du système actuel, nous présentons ci-dessous une analyse des mécanismes de contrôle existants.

La Figure 1.6 illustre le processus actuel de vérification dans OpenG2P, où l'agent social consulte manuellement les fiches des bénéficiaires et effectue des contrôles ponctuels sans outil d'aide à la décision :

## Les modules OpenG2P et la chaîne de distribution des aides sociales

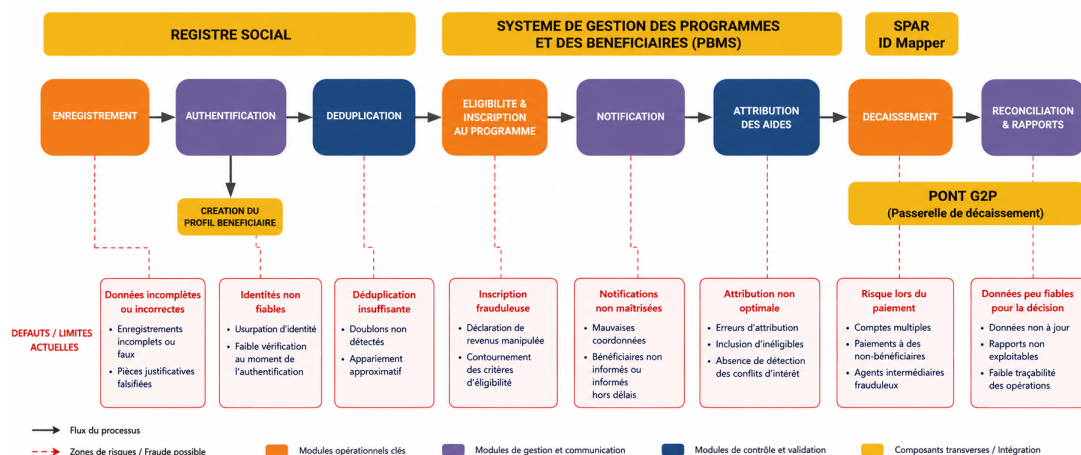


FIGURE 1.6 – Processus actuel dans OpenG2P et les zones de vulnérabilité à la fraude

Comme le montre cette figure, **aucun mécanisme de détection automatique** n'intervient entre l'enregistrement et le versement. La fraude peut s'infiltrer à chaque étape sans être détectée, et les contrôles manuels post-versement ne couvrent qu'une fraction des cas.

### 1.3.5 Objectif du projet

Face à ces constats, notre projet a pour ambition de transformer fondamentalement la manière dont les programmes de protection sociale luttent contre la fraude. L'objectif n'est pas simplement de détecter des cas suspects, mais de **changer le paradigme** : passer d'un système réactif, qui découvre la fraude après le versement, à un système intelligent qui la prévient avant qu'elle ne cause des dégâts. Pour y parvenir, nous avons structuré notre travail autour de cinq objectifs principaux :

- **Détection proactive** : identifier les bénéficiaires à risque avant le versement, au moment même de leur enregistrement dans le système.
- **Analyse multi-couches** : croiser plusieurs signaux complémentaires règles métier, apprentissage automatique, comportements inhabituels et liens entre bénéficiaires pour ne laisser aucun angle mort.
- **Explicabilité des décisions** : accompagner chaque alerte d'une explication claire en français, permettant à un agent social de comprendre et de justifier pourquoi un cas est signalé.
- **Amélioration continue** : créer un système qui apprend des retours des enquêteurs et devient plus performant au fil du temps.
- **Accessibilité et intégration** : offrir une solution 100% open-source, gratuite et intégrée directement à la plateforme OpenG2P que les gouvernements utilisent déjà.

## 1.4 Choix méthodologique et conceptuel

### 1.4.1 Méthodologie de conception

Pour la conception de ce projet, nous avons adopté le langage de modélisation **UML** afin de représenter les différentes vues du système de manière standardisée. Nous avons retenu quatre diagrammes : cas d'utilisation, classes, séquence et déploiement, qui permettent de couvrir respectivement les interactions, la structure, le flux de scoring et l'architecture Docker du système.

### 1.4.2 Méthodologie adoptée

La nature exploratoire d'un projet de data science — où les données évoluent, les modèles doivent être comparés et les hypothèses remises en question — impose une méthodologie flexible. Nous avons donc adopté la méthode **SCRUM**, un cadre agile structuré en sprints de 1 à 2 semaines, permettant de livrer des incréments fonctionnels et d'intégrer les retours d'EY Tunisia de manière continue.

#### 1.4.2.1 Comparaison entre les méthodologies traditionnelles et agiles

| Critère    | Traditionnelle                | Agile                     |
|------------|-------------------------------|---------------------------|
| Approche   | Linéaire et séquentielle      | Itérative et incrémentale |
| Exigences  | Figées au départ              | Évolutives                |
| Changement | Difficile et coûteux          | Flexible et intégré       |
| Client     | Impliqué au début et à la fin | Impliqué en continu       |
| Livraison  | Une seule fois à la fin       | Fréquente (cycles courts) |
| Risques    | Identifiés tardivement        | Maîtrisés en continu      |

TABLEAU 1.2 – Méthodologies Traditionnelles vs Agiles

#### 1.4.2.2 Pilotage du projet

Le projet a été organisé en six phases : cadrage, conception, génération de données, modélisation ML, intégration et tests. L'équipe SCRUM est composée comme suit :

**Product Owner** : EY Tunisia

**Scrum Master** : [Cyrine Garram, Mariem Abcha]

**Équipe de développement** : [Halim Trabelsi]

## 1.5 Diagramme de Gantt

Le diagramme de Gantt ci-dessous présente la planification temporelle du projet, découpée en phases et en sprints sur la durée du stage. La Figure 1.7 illustre cette répartition en utilisant les mois comme unité de temps.

FIGURE 1.7 – Diagramme de Gantt du projet

### Conclusion

Ce chapitre a posé les fondations du projet : après avoir présenté le cadre institutionnel, étudié les lacunes d'OpenG2P et du marché, et défini nos objectifs, nous avons adopté une méthodologie SCRUM adaptée à la nature itérative d'un projet de data science. Le chapitre suivant sera consacré à l'**état de l'art**, où nous examinerons les techniques de détection de fraude dans la littérature scientifique afin de justifier les choix techniques de notre solution.

# Chapitre 2 État de l'art

---

## Introduction

Avant de détailler la conception et la réalisation de notre solution, il est essentiel de poser les bases théoriques sur lesquelles elle repose. Ce chapitre a pour objectif de présenter les notions fondamentales nécessaires à la compréhension du projet et d'examiner les techniques existantes dans la littérature scientifique pour la détection de fraude.

### 2.1 Définitions et concepts clés

Il est indispensable de définir le vocabulaire et les notions fondamentales qui seront mobilisés tout au long de ce rapport. Cette section a pour but de rendre le reste du document accessible à tout lecteur, y compris ceux qui ne sont pas familiers avec les domaines de l'intelligence artificielle ou de l'analyse de données.

#### 2.1.1 Concepts liés à la fraude et à la protection sociale :

- **Programme G2P (Government-to-Person)** : mécanisme par lequel un gouvernement verse des aides financières directement aux citoyens éligibles (allocations familiales, bourses, aides alimentaires). C'est le contexte métier de notre projet.
- **Détection de fraude** : ensemble des méthodes permettant d'identifier des comportements malhonnêtes ou irréguliers dans un système. Dans notre cas, il s'agit de repérer les bénéficiaires qui tentent de recevoir indûment des aides sociales.

#### 2.1.2 Concepts d'Intelligence Artificielle et Machine Learning

L'intelligence artificielle, et plus précisément le machine learning, est au cœur de notre solution. Pour comprendre comment notre système apprend à distinguer un bénéficiaire légitime d'un fraudeur, il est nécessaire de maîtriser les notions suivantes :

- **Machine Learning (apprentissage automatique)** : branche de l'intelligence artificielle où un programme apprend à prendre des décisions à partir d'exemples passés, sans être explicitement programmé pour chaque cas. Le programme identifie des régularités dans les données et les utilise pour faire des prédictions sur de nouveaux cas.
- **Apprentissage supervisé** : type de machine learning où le modèle apprend à partir d'exemples dont on connaît déjà la réponse (par exemple : « ce bénéficiaire est un fraudeur » ou « ce bénéficiaire est légitime »). Le modèle apprend la relation entre les caractéristiques d'un bénéficiaire et son statut, puis applique cette connaissance aux nouveaux cas.
- **Apprentissage non supervisé** : type de machine learning où le modèle ne dispose pas de réponses connues. Il cherche à identifier des comportements atypiques ou des groupes

naturels dans les données, sans savoir à l'avance ce qui est normal ou anormal.

- **Feature engineering (extraction de caractéristiques)** : étape qui consiste à transformer les données brutes en variables exploitables par un modèle. Par exemple, à partir du nombre de téléphones partagés et du nombre de comptes bancaires partagés, on calcule un « score de risque réseau » qui résume l'information en un seul chiffre.
- **Data leakage (fuite de données)** : erreur dans la construction du modèle où des informations liées à la réponse (le label fraude/légitime) se retrouvent accidentellement dans les données d'entraînement. Le modèle semble alors parfait, mais ses performances sont artificiellement gonflées et ne se reproduisent pas en conditions réelles.
- **Collusion** : forme de fraude collective où plusieurs individus collaborent pour tromper le système — par exemple, en partageant un même compte bancaire ou un même numéro de téléphone pour multiplier les inscriptions.

### 2.1.3 Métriques d'Évaluation et Concepts de Validité

Pour évaluer la performance d'un modèle de détection de fraude, il est essentiel de comprendre les métriques suivantes, qui permettent de mesurer son efficacité et ses limitations :

- **Vrai positif** : cas où le système identifie correctement une transaction ou un bénéficiaire réellement frauduleux comme étant frauduleux.
- **Vrai négatif** : cas où le système identifie correctement une transaction ou un bénéficiaire légitime comme étant non frauduleux.
- **Faux positif** : cas où le système signale un bénéficiaire comme fraudeur alors qu'il est légitime. Un excès de faux positifs surcharge inutilement les enquêteurs et nuit à la confiance envers le système.
- **Faux négatif** : cas où le système laisse passer un vrai fraudeur sans le détecter. C'est la situation la plus coûteuse, car l'argent est versé à une personne qui n'y a pas droit.
- **Accuracy (exactitude)** : proportion de prédictions correctes (vrais positifs + vrais négatifs) parmi l'ensemble des prédictions effectuées.
- **Précision (precision)** : proportion de cas réellement frauduleux parmi tous les cas que le système a prédits comme frauduleux.
- **Rappel (recall)** : proportion de cas réellement frauduleux que le système a effectivement réussi à détecter parmi tous les cas frauduleux existants.
- **F1-score** : moyenne harmonique de la précision et du rappel, offrant une mesure équilibrée de la performance du modèle, surtout lorsque les classes sont déséquilibrées (comme c'est souvent le cas en détection de fraude).
- **AUC-ROC (Area Under the Receiver Operating Characteristic Curve)** : métrique d'évaluation qui mesure la capacité d'un modèle à distinguer les fraudeurs des légitimes, sur une échelle de 0 à 1. Un AUC de 0.5 signifie que le modèle ne fait pas mieux que le hasard ; un AUC de 1.0 signifie une détection parfaite. Dans la pratique, un AUC supérieur à 0.85 est considéré comme bon.

### 2.1.4 Explicabilité et génération d'explications

Un système de détection de fraude ne peut être utile que s'il est compréhensible par ses utilisateurs. Un agent social doit pouvoir expliquer à un bénéficiaire pourquoi son dossier a été signalé. C'est le rôle de l'explicabilité :

- **XAI (Explainable Artificial Intelligence)** : domaine de l'intelligence artificielle qui vise à rendre les décisions des modèles compréhensibles par des humains. Plutôt qu'une « boîte noire » qui dit simplement « fraude détectée », un système explicable indique *pourquoi* : quels facteurs ont pesé dans la décision.
- **SHAP (SHapley Additive exPlanations)** : méthode d'explicabilité qui attribue à chaque caractéristique du bénéficiaire une contribution précise au score final. Par exemple : « le partage de téléphone avec 4 autres bénéficiaires a augmenté le score de risque de +0.15 ». SHAP est fondé sur la théorie des jeux (valeurs de Shapley) et constitue aujourd'hui le standard de l'industrie.
- **LLM (Large Language Model)** : modèle d'intelligence artificielle capable de comprendre et de générer du texte en langage naturel. Dans notre projet, nous utilisons Mistral (via Ollama) pour transformer les facteurs SHAP et les drapeaux de règles en une **explication rédigée en français**, compréhensible par un agent social non-technique.
- **RAG (Retrieval-Augmented Generation)** : technique qui enrichit la réponse d'un LLM en lui fournissant des documents pertinents avant la génération. Dans notre contexte, le LLM reçoit les facteurs SHAP, les règles déclenchées et le score du bénéficiaire comme contexte, ce qui lui permet de produire une explication précise et fondée sur des faits réels, et non sur des connaissances génériques.

### 2.1.5 Analyse de graphe et détection de réseaux

La fraude n'est pas toujours l'acte d'un individu isolé. Souvent, plusieurs personnes collaborent en partageant des ressources (téléphones, comptes bancaires). L'analyse de graphe permet de révéler ces réseaux invisibles :

- **Graphe** : structure mathématique composée de **nœuds** (les entités — ici, les bénéficiaires) et d'**arêtes** (les relations entre eux — ici, le partage d'un téléphone ou d'un compte bancaire). Si deux bénéficiaires partagent le même numéro de téléphone, une arête les relie.
- **Détection de communautés** : technique qui identifie des groupes de nœuds fortement connectés entre eux. Une communauté dense de 5 à 15 bénéficiaires partageant plusieurs téléphones et comptes bancaires est un signal fort de fraude organisée.
- **Centralité (PageRank)** : mesure d'importance d'un nœud dans un réseau. Un bénéficiaire très connecté (lié à de nombreux autres bénéficiaires suspects) obtient un score de centralité élevé. Notre solution utilise l'algorithme PageRank, le même que celui inventé par Google pour classer les pages web adapté au contexte de la fraude sociale.

### 2.1.6 Modèles de Machine Learning et Architectures de Détection

Au-delà des principes fondamentaux du machine learning, plusieurs modèles et architectures spécialisés ont été développés pour traiter les problèmes complexes comme la détection de fraude. Voici les plus importants :

- **XGBoost (eXtreme Gradient Boosting)** : algorithme qui combine des centaines d'arbres de décision simples, chacun corrigeant les erreurs du précédent, pour obtenir des prédictions très précises. C'est le modèle principal retenu dans notre solution en raison de sa performance exceptionnelle sur les données déséquilibrées.
- **Random Forest (forêt aléatoire)** : algorithme qui construit plusieurs arbres de décision indépendants et combine leurs prédictions par vote majoritaire. Il est robuste au surapprentissage et sert de modèle de comparaison fiable.
- **Isolation Forest** : algorithme de détection d'anomalies qui isole les comportements atypiques en mesurant leur facilité à se séparer du reste des données. Plus un individu est isolé rapidement, plus il est suspect.
- **Régression logistique** : modèle supervisé qui estime la probabilité qu'un cas appartienne à une classe (fraude ou légitime). Bien que simple, il offre une excellente interprétabilité — chaque coefficient explique directement son impact.
- **Sur-apprentissage (overfitting)** : situation où un modèle apprend les détails spécifiques des données d'entraînement au lieu des régularités générales. Un modèle surappris performe parfaitement sur les données vues mais échoue sur les nouvelles.
- **Sous-apprentissage (underfitting)** : situation inverse où le modèle est trop simple pour capturer les régularités des données. Un modèle sous-appris performe mal sur l'entraînement et ne généralise pas aux données nouvelles.
- **Validation croisée (cross-validation)** : technique qui divise les données en plusieurs sous-ensembles et entraîne le modèle sur chaque combinaison. Elle offre une évaluation robuste et fiable de la performance réelle du modèle.
- **SHAP (SHapley Additive exPlanations)** : technique d'interprétabilité basée sur la théorie des jeux qui assigne une « contribution » à chaque feature. Elle garantit que les explications de chaque prédiction sont équitables et théoriquement justifiées.
- **Analyse de réseau et PageRank** : approche qui construit un graphe où les nœuds sont des bénéficiaires et les arêtes représentent des connexions (téléphones ou comptes partagés). PageRank attribue un score de centralité qui révèle les réseaux de fraude organisée.
- **Stacking (méta-apprentissage)** : technique qui combine les prédictions de plusieurs modèles en entraînant un méta-modèle pour fusionner les scores optimalement. Elle tire parti des forces de chaque modèle et réduit leurs faiblesses individuelles.
- **Déséquilibre de classes (class imbalance)** : problème courant en détection de fraude où les fraudes sont rares (1-5%) et les données d'entraînement contiennent beaucoup plus d'exemples légitimes. On le résout par des techniques comme SMOTE, l'ajustement des poids de classe, ou l'évaluation sur des métriques appropriées.
- **Calibration du modèle** : processus qui garantit que la probabilité prédite par le modèle (par exemple 0.7) reflète la véritable probabilité empirique de fraude. Un modèle bien calibré fournit des seuils de décision fiables pour les systèmes de production.

Ces définitions posées, nous disposons maintenant du vocabulaire nécessaire pour examiner en détail chacune des techniques de détection de fraude dans les sections suivantes.

### 2.1.7 Identification des acteurs

Notre système de détection de fraude interagit avec plusieurs acteurs, qu'ils soient des utilisateurs humains ou des systèmes informatiques. L'identification de ces acteurs est une étape essentielle pour définir les besoins fonctionnels de la solution. Le Tableau 2.1 présente les acteurs internes qui utilisent directement le système, tandis que le Tableau 2.2 décrit les acteurs externes et les systèmes avec lesquels notre solution communique.

| Acteur          | Rôle                                                                                                                                                                                                                        |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bénéficiaire    | Personne inscrite au programme de protection sociale qui reçoit (ou tente de recevoir) les aides. C'est l'entité analysée par le moteur de détection de fraude.                                                             |
| Agent social    | Personnel chargé de l'enregistrement des bénéficiaires dans OpenG2P et de la vérification des informations sur le terrain. Il est le premier point de contact avec le bénéficiaire.                                         |
| Analyste fraude | Utilisateur qui consulte les alertes générées par le système, examine les explications fournies (SHAP + LLM) et décide des actions à entreprendre : validation, investigation ou blocage du versement.                      |
| Administrateur  | Responsable de la configuration des règles métier, des seuils de détection et de la gestion technique de la plateforme (utilisateurs, droits d'accès, paramètres ML).                                                       |
| Superviseur     | Responsable qui consulte les tableaux de bord pour suivre les indicateurs globaux (taux de fraude, performance du modèle, tendances), du réentraînement des modèles et prendre des décisions stratégiques sur le programme. |

TABLEAU 2.1 – Acteurs internes du système

| Acteur / Système                          | Rôle                                                                                                                                                                                   |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OpenG2P / Odoo                            | Plateforme source qui gère le registre des bénéficiaires, les programmes et les paiements. Elle transmet les données au moteur de détection et reçoit les alertes de fraude en retour. |
| Moteur intelligent de détection de fraude | Système central qui reçoit les données, applique les quatre couches d'analyse (règles, ML, graphe, anomalie), produit un score de risque et génère une explication en langage naturel. |
| Organisme de protection sociale           | Entité institutionnelle (ministère, agence gouvernementale) bénéficiaire des rapports de fraude, responsable des politiques publiques et des décisions budgétaires.                    |

TABLEAU 2.2 – Acteurs externes et systèmes

### 2.1.8 Présentation des travaux existants

Avant de développer notre solution technologique, il est essentiel de réaliser un état de l'art des travaux existants dans le domaine de la détection de la fraude sociale. Les tableaux suivants présentent une synthèse de ce panorama sectoriel. Pour plus de clarté, la littérature académique, les solutions commerciales et les architectures DPI ont été séparées. Cette classification permet de mettre en lumière les leviers techniques ainsi que les méthodes analytiques qui caractérisent chaque approche.

TABLEAU 2.3 – Comparaison des méthodes de détection de fraude : avantages et limites

| Approche                                        | Principe                                                                              | Avantages                                                                                                                                                                                   | Limites                                                                                                                                                                   |
|-------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Règles métier (rule-based)</b>               | Conditions explicites définies par des experts métier (seuils, combinaisons logiques) | <ul style="list-style-type: none"> <li>+ Transparent et facile à auditer</li> <li>+ Exécution rapide, sans entraînement</li> <li>+ Pas besoin de données labellisées</li> </ul>             | <ul style="list-style-type: none"> <li>– Rigide face aux nouvelles fraudes</li> <li>– Maintenance manuelle lourde</li> <li>– Ne détecte que les schémas connus</li> </ul> |
| <b>ML supervisé (XGBoost, RF)</b>               | Apprentissage à partir d'exemples étiquetés fraude / légitime                         | <ul style="list-style-type: none"> <li>+ Détecte des schémas subtils et complexes</li> <li>+ Bonne précision globale (AUC élevé)</li> <li>+ S'améliore avec le volume de données</li> </ul> | <ul style="list-style-type: none"> <li>– Nécessite des données labellisées</li> <li>– Risque de fuite de données</li> <li>– Peu explicable par défaut</li> </ul>          |
| <b>Détection d'anomalies (Isolation Forest)</b> | Identification de comportements atypiques sans étiquettes, par isolement statistique  | <ul style="list-style-type: none"> <li>+ Détecte la fraude inconnue</li> <li>+ Aucun label requis (non supervisé)</li> <li>+ Complémentaire au ML supervisé</li> </ul>                      | <ul style="list-style-type: none"> <li>– Taux de faux positifs élevé</li> <li>– Difficile à interpréter</li> <li>– Sensible au paramétrage</li> </ul>                     |
| <b>Analyse de réseau (PageRank)</b>             | Graphe de liens entre bénéficiaires (téléphone, compte bancaire partagé)              | <ul style="list-style-type: none"> <li>+ Révèle la fraude organisée</li> <li>+ Capture la collusion invisible au ML</li> <li>+ Vision relationnelle unique</li> </ul>                       | <ul style="list-style-type: none"> <li>– Coûteux en calcul</li> <li>– Dépend de la qualité des liens</li> <li>– Complexe à mettre en place</li> </ul>                     |

TABLEAU 2.4 – Solutions commerciales de détection de fraude sur le marché

| Solution                         | Principe / Utilité                                                                                                                    | Avantages                                                                                                                                                                                                      | Limites                                                                                                                                                              |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FICO Falcon Fraud Manager</b> | Scoring en temps réel basé sur un consortium de données partagées entre banques et un moteur ML supervisé.                            | <ul style="list-style-type: none"> <li>+ Consortium mondial (9 000+ institutions)</li> <li>+ Détection temps réel à très haute échelle</li> <li>+ AUC <math>\sim 0.95</math> sur la fraude bancaire</li> </ul> | <ul style="list-style-type: none"> <li>– Coût : 100k–500k+\$/an</li> <li>– Conçu pour la fraude bancaire uniquement</li> <li>– Code propriétaire fermé</li> </ul>    |
| <b>Feedzai (RiskOps)</b>         | Plateforme IA-native combinant ML supervisé et analyse de graphe pour le scoring de transactions en temps réel.                       | <ul style="list-style-type: none"> <li>+ Analyse de graphe intégrée</li> <li>+ Architecture API-first moderne</li> <li>+ Adaptable à plusieurs canaux de paiement</li> </ul>                                   | <ul style="list-style-type: none"> <li>– Coût : 150k–600k+\$/an</li> <li>– Ciblé paiements / fin-tech</li> <li>– Pas d'intégration G2P possible</li> </ul>           |
| <b>SAS Fraud Management</b>      | Suite analytique avancée combinant des modèles ensemblistes, des alertes configurables et un moteur d'investigation.                  | <ul style="list-style-type: none"> <li>+ Couverture multi-secteurs</li> <li>+ Moteur d'investigation intégré</li> <li>+ Plateforme cloud SAS Viya</li> </ul>                                                   | <ul style="list-style-type: none"> <li>– Coût : 200k–1M+\$/an</li> <li>– Intégration sur mesure complexe</li> <li>– Explicabilité limitée pour le terrain</li> </ul> |
| <b>NICE Actimize</b>             | Surveillance des transactions et conformité anti-blanchiment (AML) avec workflows d'enquête intégrés.                                 | <ul style="list-style-type: none"> <li>+ Leader en conformité réglementaire</li> <li>+ Workflows d'investigation complets</li> <li>+ Couverture AML et fraude combinée</li> </ul>                              | <ul style="list-style-type: none"> <li>– Coût : 300k–2M+\$/an</li> <li>– Aucune analyse de réseau (graphe)</li> <li>– Dépendance éditeur très forte</li> </ul>       |
| <b>Featurespace (ARIC)</b>       | Analyse comportementale adaptative qui modélise le comportement normal de chaque utilisateur et détecte les déviations en temps réel. | <ul style="list-style-type: none"> <li>+ Détection non supervisée innovante</li> <li>+ S'adapte sans réentraînement</li> <li>+ XAI avec reason codes</li> </ul>                                                | <ul style="list-style-type: none"> <li>– Coût : 100k–400k\$/an</li> <li>– Ciblé banque et assurance</li> <li>– Pas d'analyse de graphe</li> </ul>                    |

TABLEAU 2.5 – Limites des travaux existants face à la philosophie DPI / DPG

| Dimension                               | Ce qui manque dans les solutions actuelles                                                                                                | Problème non résolu face au DPI / DPG                                                                                                          |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Accessibilité financière</b>         | Les solutions commerciales coûtent de <b>100k\$ à plus de 2M\$ par an</b> , hors de portée des budgets publics en développement.          | <b>Contredit l'objectif d'inclusion des DPG</b> : une technologie réservée aux pays riches reproduit les inégalités qu'elle prétend combattre. |
| <b>Ouverture du code</b>                | <b>Les moteurs de détection sont propriétaires et fermés</b> , impossible de les auditer, les adapter ou les réutiliser.                  | <b>Va à l'encontre du principe fondamental des Biens Publics Numériques</b> : code ouvert, réutilisable et vérifiable par tous.                |
| <b>Souveraineté des données</b>         | Les plateformes cloud propriétaires imposent que <b>les données sensibles transitent par des serveurs externes</b> .                      | <b>Menace la souveraineté numérique des États</b> : les données des bénéficiaires doivent rester sous contrôle national.                       |
| <b>Intégration native OpenG2P</b>       | Aucune solution du marché n'est conçue pour s'intégrer nativement à OpenG2P ou aux registres sociaux.                                     | <b>Oblige des développements sur mesure coûteux</b> : rompt l'interopérabilité modulaire promue par l'écosystème DPI.                          |
| <b>Spécificité de la fraude sociale</b> | <b>Les outils ciblent la fraude bancaire</b> (carte, paiement) et non les fraudes propres au social : multi-inscription, ménages fictifs. | <b>Les schémas de fraude G2P (collusion, partage d'identité)</b> restent mal couverts par des modèles pensés pour la finance.                  |
| <b>Explicabilité pour le terrain</b>    | <b>Les scores sont produits par des boîtes noires</b> , sans justification compréhensible par un agent social non-technique.              | <b>Empêche la redevabilité exigée dans les services publics</b> : une décision d'aide doit pouvoir être expliquée au citoyen.                  |
| <b>Proactivité (avant versement)</b>    | <b>Les contrôles traditionnels (audits manuels)</b> interviennent après le versement, quand la perte est déjà subie.                      | <b>Le modèle DPI vise une prévention en amont</b> : l'absence de détection temps réel laisse fuir les budgets avant toute action.              |

## Conclusion

L'examen des travaux et des outils actuels montre que chacune des approches explorées apporte une contribution utile mais reste limitée lorsqu'elle est appliquée isolément aux programmes de protection sociale du contexte des DPI et DPG, spécifiquement OpenG2P. À partir de ces constats, nous élaborons dans le chapitre suivant une spécification détaillée des exigences fonctionnelles et une architecture technologique hybride capable de surmonter ces limitations.

# Chapitre 3 Spécifications fonctionnelles et choix technologiques

---

## 3.1 Introduction

Forts des constats du chapitre précédent, l'absence de solution satisfaisant simultanément les exigences d'accessibilité, de transparence et de souveraineté. Nous orientons notre conception vers une approche systématique et traçable. Ce chapitre traduit ces apprentissages en spécifications formelles qui guident la construction de notre moteur intelligent de détection de fraude.

## 3.2 Identification des besoins fonctionnelles

TABEAU 3.1 – Spécification des besoins fonctionnels du moteur intelligent de détection de fraude par acteur

| ID                                                | Acteur          | Besoin fonctionnel                                                                                                                                |
|---------------------------------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| AGENT SOCIAL — Analyse unitaire                   |                 |                                                                                                                                                   |
| BF01                                              | Agent social    | <b>Déclencher un scan unitaire</b> : lancer manuellement l'analyse de fraude sur un bénéficiaire et consulter le score de risque.                 |
| BF01                                              | Agent social    | <b>Consulter le score de risque</b> : Consulter si le bénéficiaire est classé LOW, MEDIUM, HIGH ou CRITICAL et visualiser les règles déclenchées. |
| ANALYSTE FRAUDE — Examiner et valider les alertes |                 |                                                                                                                                                   |
| BF02                                              | Analyste fraude | <b>Consulter les alertes de fraude</b> : visualiser liste des cas détectés avec score, niveau de risque et règles déclenchées.                    |
| BF03                                              | Analyste fraude | <b>Examiner les explications</b> : consulter facteurs SHAP et explication en français générée par le LLM.                                         |
| BF04                                              | Analyste fraude | <b>Donner un feedback</b> : valider ou infirmer une alerte (fraude confirmée, faux positif, indéterminé) pour boucle d'amélioration.              |
| BF05                                              | Analyste fraude | <b>Exporter un rapport d'investigation</b> : générer rapport en PDF ou CSV pour un cas de fraude.                                                 |
| BF06                                              | Analyste fraude | <b>Versionner les modèles</b> : consulter historique MLflow, comparer performances et effectuer un rollback.                                      |

| ID                                           | Acteur      | Besoin fonctionnel                                                                                                                             |
|----------------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ADMINISTRATEUR — Configurer le moteur</b> |             |                                                                                                                                                |
| BF07                                         | Admin       | <b>Gérer les règles métier</b> : ajouter, modifier, activer/désactiver règles de détection (seuils, poids, conditions).                        |
| BF08                                         | Admin       | <b>Recharger les règles en temps réel</b> : appliquer modifications sans redémarrer le système.                                                |
| BF09                                         | Admin       | <b>Réentraîner le modèle ML</b> : déclencher réentraînement avec nouvelles données et feedbacks collectés.                                     |
| <b>SUPERVISEUR — Monitorer et décider</b>    |             |                                                                                                                                                |
| BF10                                         | Superviseur | <b>Consulter le tableau de bord</b> : visualiser indicateurs (nombre cas, répartition par risque, tendances, taux faux positifs).              |
| BF10                                         | Superviseur | <b>Bloquer ou authentifier les bénéficiaires</b> : visualiser indicateurs (nombre cas, répartition par risque, tendances, taux faux positifs). |
| BF10                                         | Superviseur | <b>Suivre la performance du modèle</b> : consulter métriques ML (AUC, précision, rappel) et détecter dégradation.                              |
| <b>Traiter et générer automatiquement</b>    |             |                                                                                                                                                |
| BF12                                         | Système     | <b>Scanner en masse</b> : analyser automatiquement l'ensemble des bénéficiaires en un seul appel (scoring batch).                              |
| BF13                                         | Système     | <b>Scoring temps réel</b> : déclencher automatiquement l'analyse à chaque nouvel enregistrement via webhook.                                   |
| BF14                                         | Système     | <b>Calculer le score hybride</b> : combiner quatre sous-scores (règles, ML, anomalie, graphe) en score final unique.                           |
| BF15                                         | Système     | <b>Générer l'explication</b> : produire automatiquement facteurs SHAP et explication en français via LLM Mistral.                              |
| BF16                                         | Système     | <b>Synchroniser avec Odoo</b> : créer automatiquement cas de fraude dans module Odoo et mettre à jour leur statut.                             |
| BF17                                         | Système     | <b>Envoyer des alertes</b> : notifier analystes lorsqu'un cas CRITICAL ou HIGH est détecté.                                                    |

### 3.3 Identification des besoins non fonctionnels

Les besoins non fonctionnels concernent les contraintes à prendre en considération pour mettre en place une solution adéquate aux attentes des clients. Notre plateforme doit nécessairement assurer ces besoins :

- **Sécurité** : La plateforme doit garantir la confidentialité et l'intégrité des données personnelles des bénéficiaires, en conformité avec les réglementations en vigueur (ex : RGPD)
- **Scalabilité** : La solution doit être capable de gérer un nombre croissant de bénéficiaires et de transactions sans dégradation des performances.
- **Accessibilité** : L'interface utilisateur doit être intuitive et accessible à des agents sociaux ayant des compétences techniques limitées.
- **Interopérabilité** : Le moteur de détection doit s'intégrer facilement avec d'autres systèmes utilisés par les organismes de protection sociale.
- **Maintenabilité** : Le code doit être bien structuré, documenté et modulaire pour faciliter les mises à jour et les évolutions futures.

### 3.4 Diagramme de cas d'utilisation général

Le diagramme de cas d'utilisation est le meilleur outils pour décrire les interactions entre nos acteurs et notre système étudié. Il représente les différents cas d'utilisation du système, en décrivant les scénarios dans lesquels les acteurs interagissent avec celui-ci :

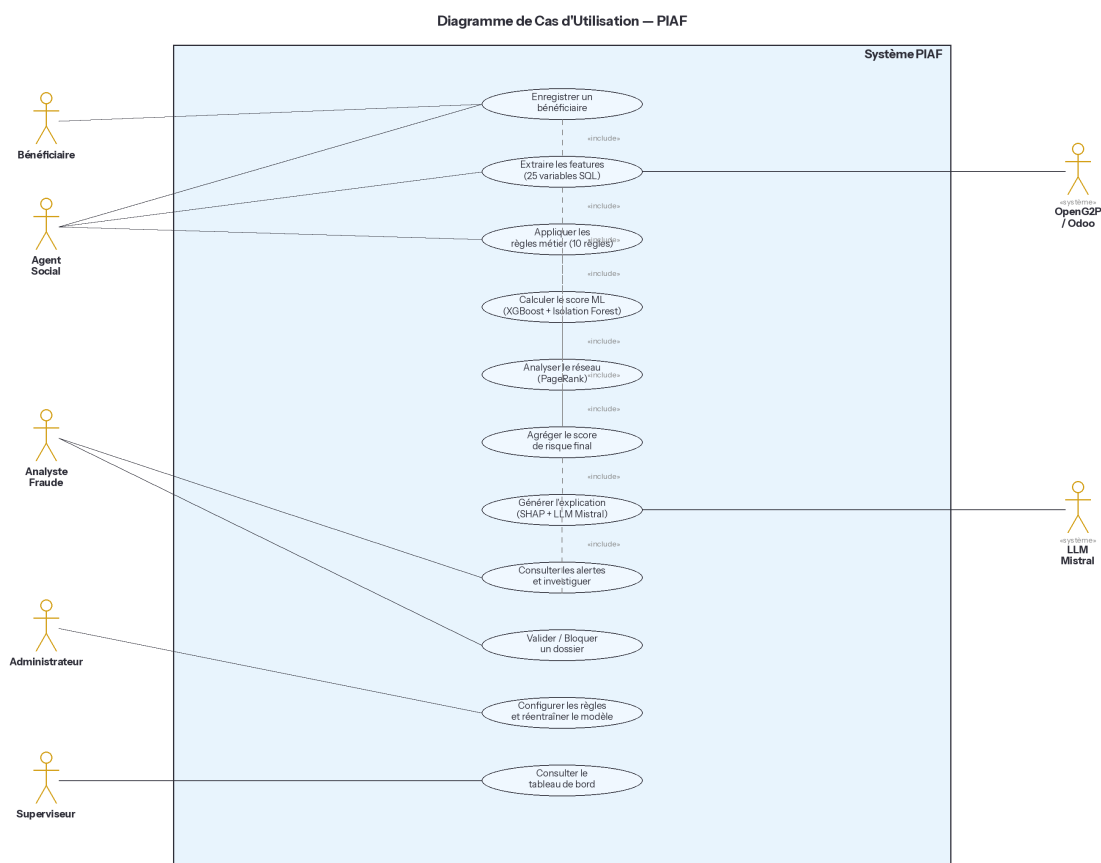


FIGURE 3.1 – Diagramme de cas d'utilisation général

### 3.5 Diagramme de classes général

Le diagramme de classes est l'un des outils de modélisation les plus efficaces pour représenter la structure statique d'un système. Il permet de décrire les différentes classes du système révélant leurs attributs définissant aussi les relations entre elles : l'héritage, l'association, l'agré-gation ou la composition. Il procure une vue d'ensemble de l'agencement et de l'organisation interne du système étudié. :

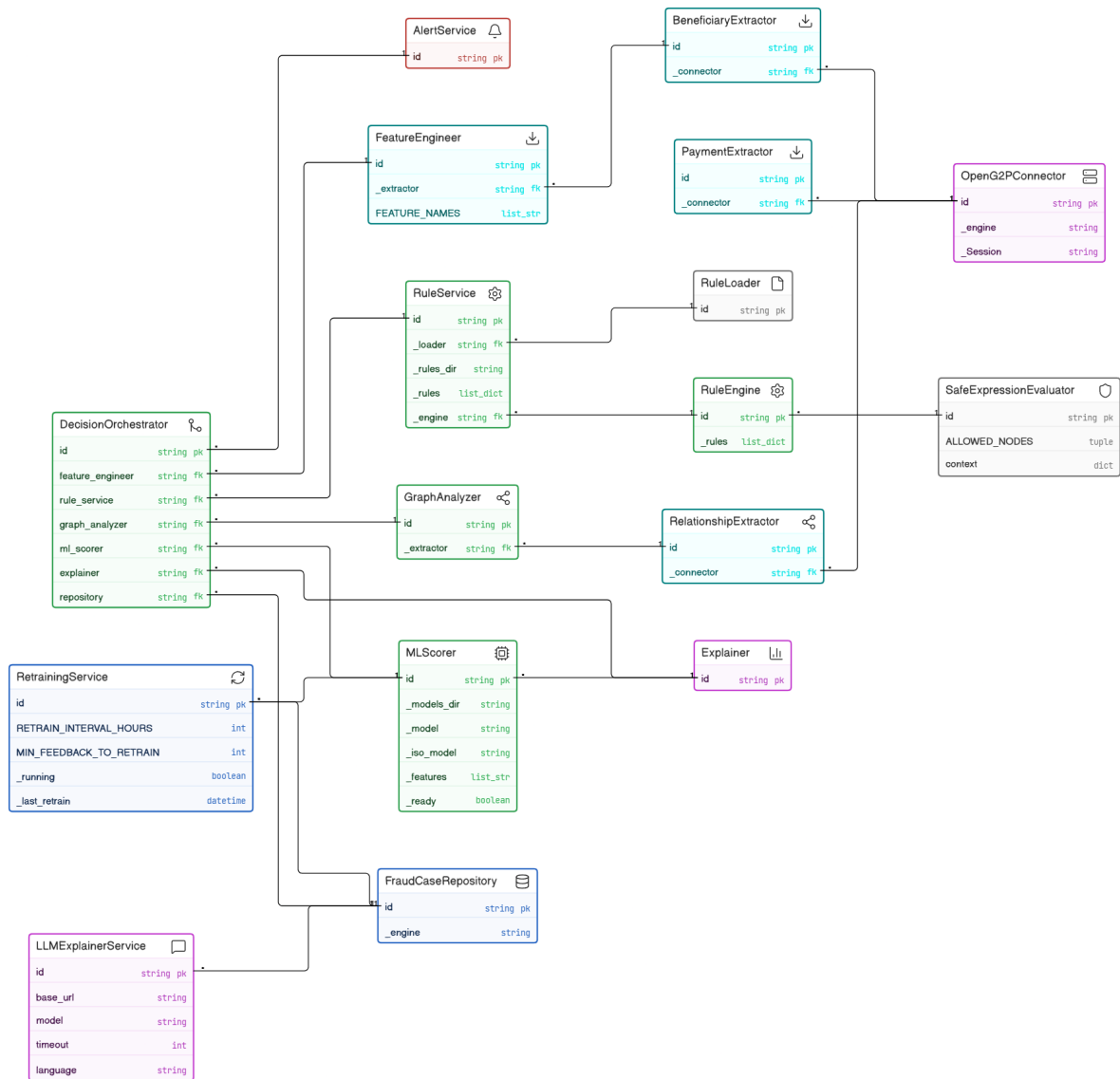


FIGURE 3.2 – Diagramme de classe général

### 3.6 Architecture physique

Cette section présente l'architecture matérielle et logicielle de l'application, en décrivant la mise en œuvre concrète des différents nœuds qui composent le système. La configuration physique, telle qu'illustrée sur le schéma, établit une distinction claire entre le poste de travail de l'utilisateur, les serveurs d'application, le serveur de persistance et les services distants.

Le système s'articule autour des composants physiques suivants :

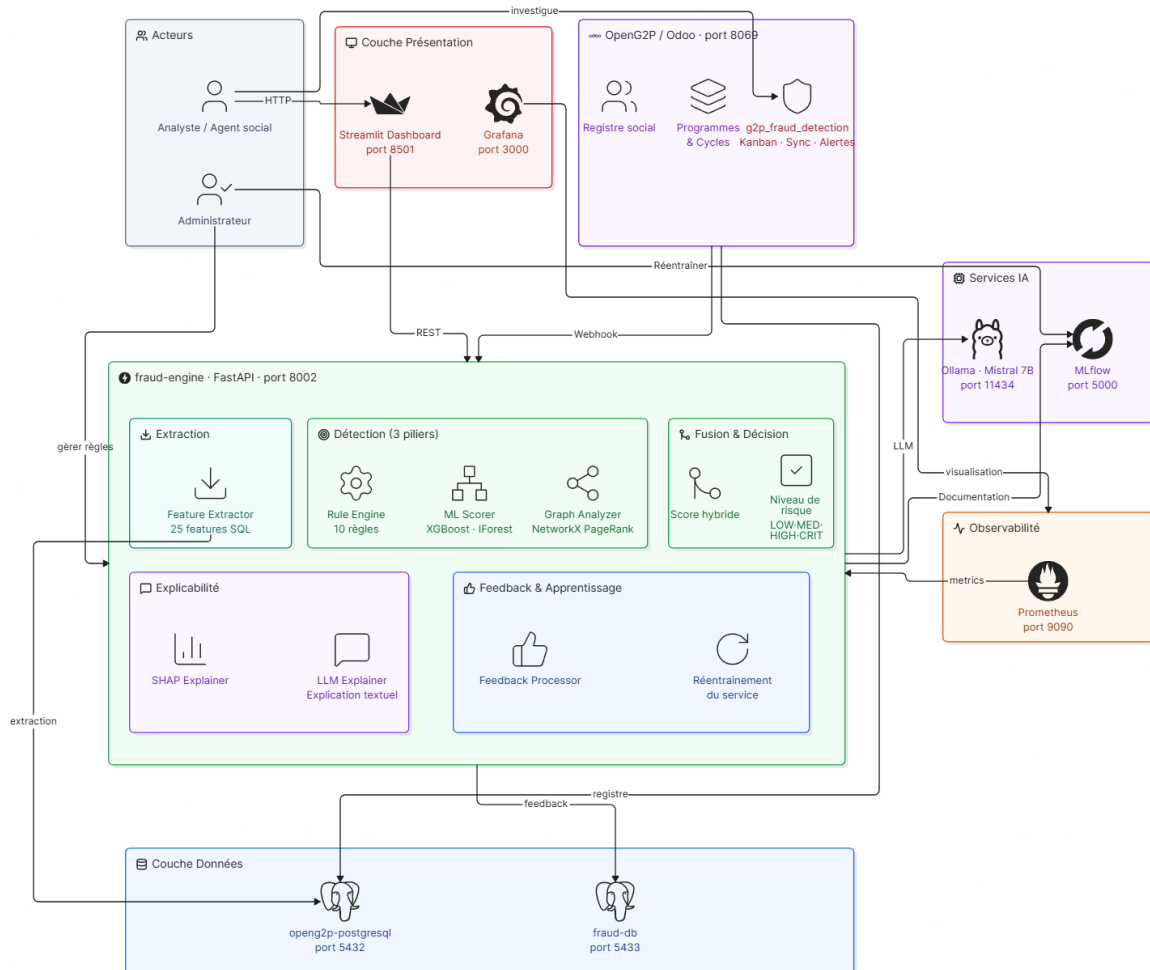


FIGURE 3.3 – Architecture physique du moteur intelligent de détection de fraude

### 3.7 Architecture logique

Dans cette section, nous allons présenter les technologies, les frameworks et les composants matériels que nous avons choisis pour notre application. Notre application est basée sur une architecture client–serveur distribuée composée de plusieurs machines logiques : client, serveur d’application, serveur d’intelligence artificielle et serveur de base de données.

Le système est constitué des éléments suivants :

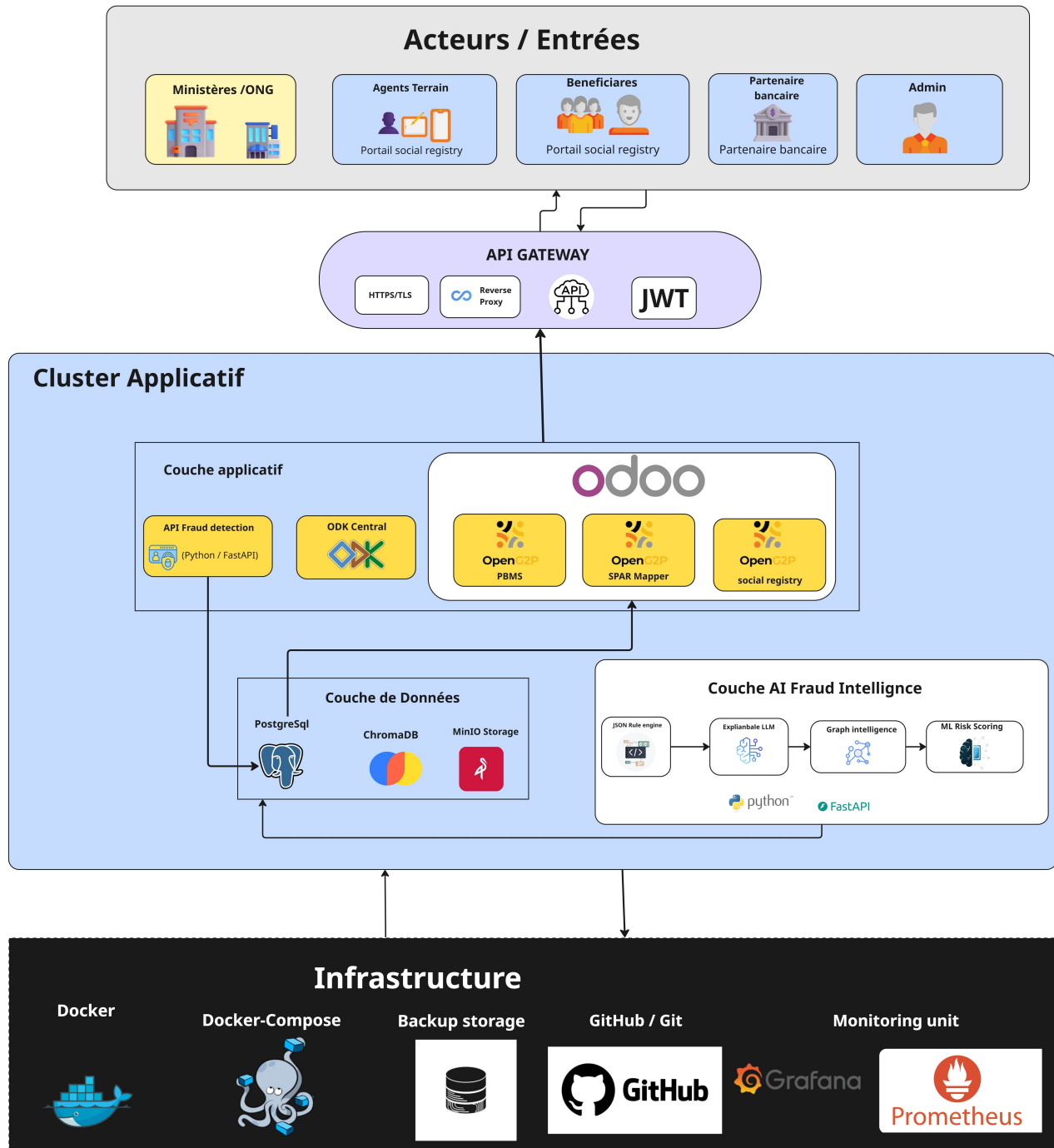


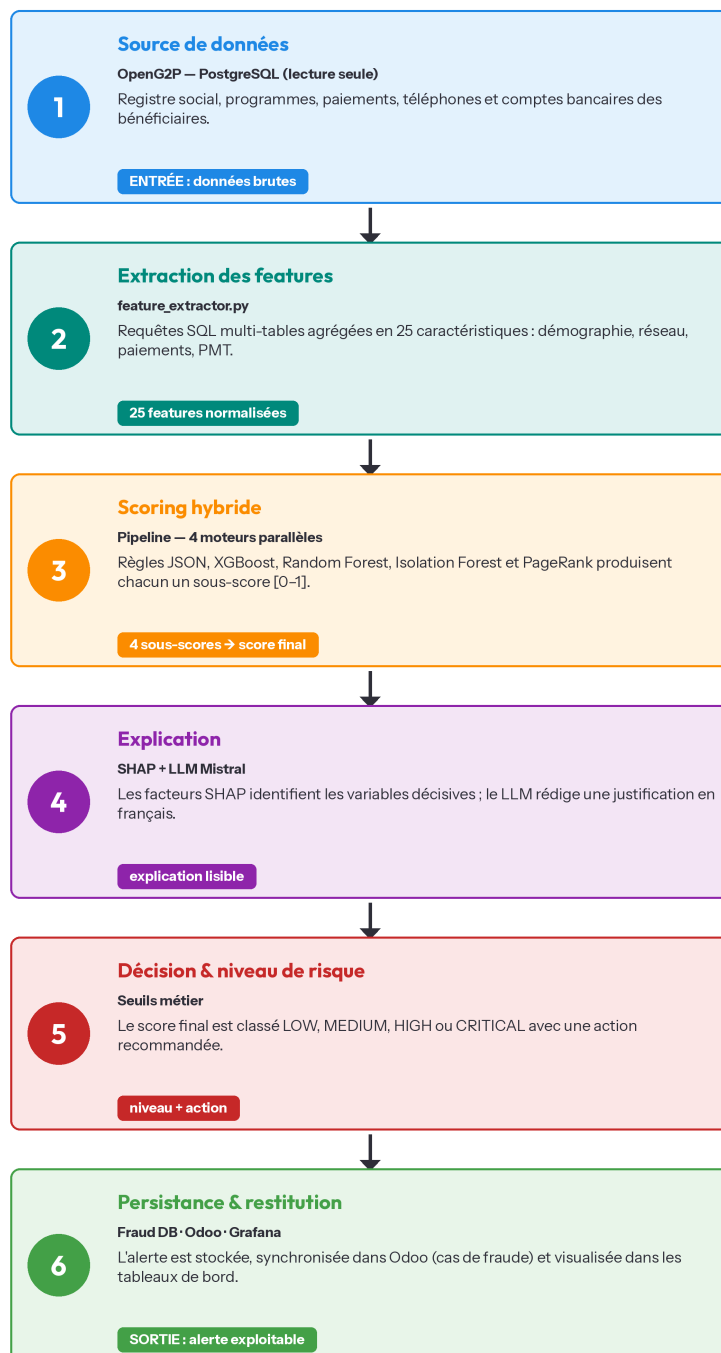
FIGURE 3.4 – Architecture logique du moteur intelligent de détection de fraude

### 3.8 Organisation de flux de données

Le diagramme de flux de données (DFD) est un outil de modélisation qui permet de représenter les flux d'informations au sein d'un système. Il illustre comment les données circulent entre les différents composants du système, en mettant en évidence les processus, les entités externes et les magasins de données :

## Organisation des flux de données

Pipeline PIAF : du bénéficiaire OpenG2P à l'alerte de fraude restituée



*Le flux est unidirectionnel et traçable : chaque étape enrichit la donnée sans jamais modifier la source OpenG2P, garantissant l'intégrité du registre social et la séparation stricte lecture / écriture.*

FIGURE 3.5 – Diagramme de flux de données du moteur intelligent de détection de fraude

### 3.9 Spécifications techniques et environnement logiciel

Après avoir défini les besoins fonctionnels et non fonctionnels de notre moteur de détection de fraude, nous devons maintenant présenter le stack technologique sur lequel repose l'implémentation. Cette section détaille les choix architecturaux et les outils logiciels qui nous ont permis de construire une solution robuste, explicable et facilement maintenable.

Notre solution s'appuie sur une architecture hybride et modulaire, combinant des composants d'intelligence artificielle (apprentissage automatique, graphes, explicabilité), des règles métier paramétrables, et une infrastructure cloud-native. Cette approche multicouche nous permet de couvrir un maximum de scénarios de fraude tout en restant auditable et transparentes vis-à-vis des parties prenantes.

### 3.10 Stack de développement backend et données

| Composant       | Technologie / Description                                                                                                                                                                                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Base de données | <b>PostgreSQL</b> : Système de gestion de base de données relationnelle open source hautement fiable, avec support complet du SQL et des transactions ACID. Utilisé pour persister les données d'enregistrement des bénéficiaires, les alertes de fraude et l'historique des décisions. |
| Système intégré | <b>OpenG2P / Odoo 17</b> : Plateforme open-source de gestion de programmes de protection sociale. Manage l'enregistrement des bénéficiaires, l'administration des programmes d'aide et l'intégration de notre moteur de détection de fraude.                                            |
| ML et graphe    | <b>Python (scikit-learn, XGBoost, NetworkX, IsolationForest, RandomForest, Linear Regression)</b> : Écosystème utilisé pour développer les modèles de machine learning, l'extraction de caractéristiques et l'analyse de graphe.                                                        |

TABLEAU 3.2 – Technologies de base du système de détection de fraude

Au-delà de ces composants fondamentaux, notre architecture s'appuie sur un ensemble d'outils dédiés à l'explicabilité, au monitoring et à la gestion du cycle de vie du modèle :

| Couche              | Outil / Technologie                                                                                                                                                                                                                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Explicabilité       | <b>SHAP</b> : Bibliothèque Python pour l'interprétabilité calculant la contribution de chaque caractéristique au score de fraude. <b>Mistral via Ollama</b> : Modèle de langue généraliste exécuté localement pour transformer les coefficients SHAP et règles en explications textuelles en français. |
| Analyse de graphe   | <b>Neo4j</b> : Base de données graphe pour stocker et analyser les réseaux de bénéficiaires (partage de téléphones, comptes bancaires). Permet la détection de communautés frauduleuses et le calcul de centralité (PageRank).                                                                         |
| Gestion des modèles | <b>MLflow</b> : Plateforme pour la gestion du cycle de vie des modèles ML. Enregistre les hyperparamètres, métriques de performance (AUC, précision, rappel) et permet un rollback facilité.                                                                                                           |

TABLEAU 3.3 – Technologies d'explicabilité et de gestion des modèles

### 3.11 Stack de développement et DevOps

Pour assurer la qualité, la maintenabilité et le déploiement continu de notre solution, nous avons intégré des outils modernes de développement et d'automatisation :

| Catégorie          | Outil / Framework                                                                                                                                                                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Éditeur de code    | <b>Visual Studio Code</b> : Éditeur léger offrant le débogage intégré, autocomplétion intelligente, intégration native de Git et extensions pour Python, SQL et Docker.                                                                                                                                                     |
| Gestion de version | <b>Git / GitHub</b> : Système de contrôle de version pour la collaboration, suivi des changements et gestion des branches. GitHub héberge le dépôt avec workflows d'intégration continue.                                                                                                                                   |
| Containerisation   | <b>Docker</b> : Plateforme de containerisation pour packager l'application avec toutes ses dépendances. Assure la cohérence entre développement, test et production.                                                                                                                                                        |
| Orchestration      | <b>Docker Compose</b> : Outil d'orchestration de conteneurs permettant de définir et de lancer l'ensemble de l'infrastructure applicative (backend, base de données, services auxiliaires) via un fichier de configuration YAML unique. Facilite les déploiements reproductibles et le développement local multi-conteneur. |

TABLEAU 3.4 – Outils de développement et d'infrastructure

#### 3.11.1 Dashboards et visualisation des résultats

Au-delà de la détection elle-même, il est essentiel de fournir aux différents utilisateurs (analystes fraude, superviseurs, administrateurs) des interfaces de visualisation intuitive et interactive. Notre solution intègre plusieurs couches de dashboards et de reporting :

| <b>Dashboard /<br/>Plateforme</b>                  | <b>Description et cas d'usage</b>                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Streamlit - Inter-<br/>face analytique</b>      | Plateforme web interactive développée en Python permettant aux analystes fraude et superviseurs de consulter les alertes détectées, visualiser les explications SHAP en temps réel, et accéder à des statistiques globales de fraude. Le dashboard Streamlit offre une réactivité immédiate et permet des filtres dynamiques (par zone géographique, programme, niveau de risque). |
| <b>MLflow - Suivi<br/>des modèles</b>              | Tableau de bord dédié au versionnage et au monitoring des modèles ML. Permet de comparer les performances de différentes versions du modèle (AUC, précision, rappel), de visualiser les métriques d'entraînement, et de tracer la provenance de chaque artefact ML. Essential pour la gouvernance et la reproductibilité.                                                          |
| <b>Heatmap géo-<br/>graphique</b>                  | Visualisation cartographique interactive montrant la concentration géographique des alertes de fraude détectées. Chaque zone est colorée proportionnellement au nombre et à la sévérité des cas signalés, permettant aux superviseurs d'identifier rapidement les hotspots de fraude et d'adapter les stratégies d'investigation par région.                                       |
| <b>Tableau de bord<br/>superviseur</b>             | Vue synthétique affichant les KPI globaux : nombre total de cas analysés, taux de fraude détecté, proportion de vrais positifs, tendances temporelles, distribution par niveau de risque, et état des investigations en cours. Utilisé par les décideurs pour le monitoring stratégique du programme.                                                                              |
| <b>Système de re-<br/>porting automa-<br/>tisé</b> | Génération automatique de rapports PDF ou CSV consolidant les résultats de détection, les explications SHAP, les analyses de réseau et les recommandations d'action pour chaque bénéficiaire signalé. Ces rapports sont exportables pour audit externe ou communication auprès du management.                                                                                      |

TABLEAU 3.5 – Dashboards et plateformes de visualisation

Ces outils de visualisation sont complémentaires aux couches techniques : tandis que MLflow assure la traçabilité des modèles côté développement, Streamlit et les dashboards superviseur fournissent des interfaces métier accessibles aux non-techniciens. La heatmap géographique facilite le diagnostic spatial de la fraude, et le système de reporting garantit que chaque décision prise peut être justifiée et audité.

### 3.11.2 Collaboration et documentation

Enfin, pour faciliter la communication entre les membres de l'équipe et maintenir une documentation à jour :

| Fonction                         | Outil                                                                                                                                                                                                                                                             |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tests et validation d'API</b> | <b>Postman / Swagger</b> : Clients HTTP interactifs pour tester manuellement et automatiquement l'API REST du moteur de fraude. Permettent de vérifier les endpoints, les paramètres, les codes de retour et de générer de la documentation API automatique.      |
| <b>Monitoring opérationnel</b>   | <b>ELK Stack (Elasticsearch, Logstash, Kibana)</b> : Centralisation et visualisation en temps réel des logs de l'application. Permet de détecter rapidement les erreurs en production, d'analyser les tendances de performance et de diagnostiquer les problèmes. |
| <b>Documentation technique</b>   | <b>Confluence / Wiki GitHub</b> : Dépôt centralisé pour les guides d'installation, les diagrammes d'architecture, les spécifications API complètes, les tutoriels d'utilisation et l'historique des décisions techniques.                                         |

TABLEAU 3.6 – Outils de collaboration et de documentation

Cet ensemble complet d'outils et de technologies forme un écosystème cohérent et bien articulé. Chaque composant remplit un rôle spécifique : les technologies de base assurent la performance et la fiabilité, les outils d'explicabilité garantissent la transparence, les dashboards et visualisations rendent les résultats accessibles aux utilisateurs métier, et enfin les outils de collaboration et documentation maintiennent la qualité et la traçabilité du projet.

## 3.12 Conclusion

Ce chapitre a consolidé les fondations techniques et conceptuelles de notre moteur de détection de fraude pour les programmes de protection sociale. En commençant par une analyse détaillée des acteurs internes et externes, nous avons défini les besoins fonctionnels et non fonctionnels qui doivent guider l'implémentation.

# Chapitre 4 Réalisation

## Introduction

Le chapitre précédent a posé les bases du système : nous y avons défini les besoins fonctionnels, justifié les choix technologiques et modélisé les flux de données. Ce chapitre passe à la mise en œuvre concrète. Il suit le parcours réel de la donnée à travers le système : depuis son extraction des tables PostgreSQL d'OpenG2P jusqu'à la décision de fraude finale restituée à l'agent social. Chaque section répond à une question d'ingénierie précise : pourquoi cette étape est nécessaire, comment elle a été implémentée, et quel résultat elle produit.

## 4.1 Environnement de développement

Le développement du moteur intelligent s'appuie sur un ensemble de technologies assurant le traitement des données, l'intelligence artificielle, le stockage et le déploiement de la solution. La Figure 4.1 présente les principaux langages, frameworks et outils utilisés ainsi que leur rôle dans le projet.

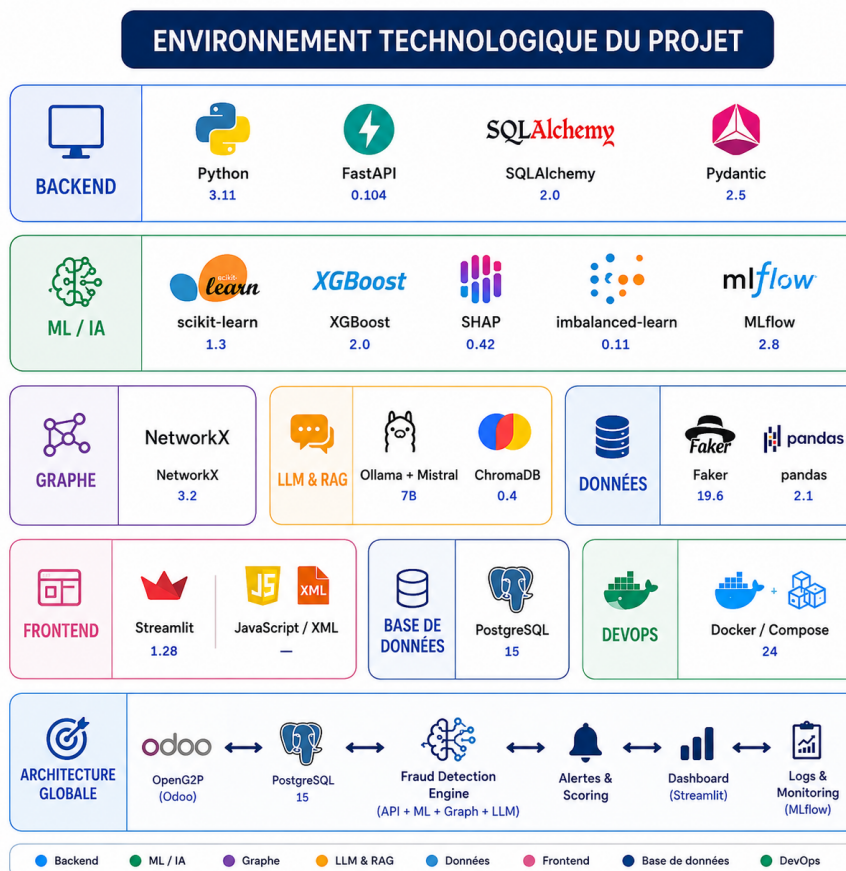


FIGURE 4.1 – Langages et outils utilisés pour le développement du moteur intelligent de détection de fraude

4.1.1 Configuration Docker Compose

L'ensemble du système fonctionne via Docker Compose, ce qui garantit un déploiement reproductible et isolé. Le Tableau 4.1 détaille les six conteneurs orchestrés, leur port d'exposition, l'image de base utilisée et leur rôle respectif dans l'architecture globale.

TABLEAU 4.1 – Conteneurs Docker du système

| Conteneur    | Port  | Image / Base         | Rôle                                        |
|--------------|-------|----------------------|---------------------------------------------|
| fraud-engine | 8002  | Python 3.11 (custom) | API FastAPI — moteur de détection de fraude |
| odoo         | 8069  | openG2P/odoo:17      | Plateforme sociale OpenG2P                  |
| postgresql   | 5432  | postgres:15          | Base partagée (OpenG2P + Fraud DB)          |
| streamlit    | 8501  | Python 3.11 (custom) | Dashboard alertes et monitoring             |
| mlflow       | 5000  | ghcr.io/mlflow       | Suivi des expériences ML                    |
| ollama       | 11434 | ollama/ollama        | Serveur LLM local (Mistral 7B)              |

La Figure 4.2 présente un extrait de la configuration Docker Compose pour les deux services principaux du système, illustrant la manière dont ces conteneurs sont déclarés, reliés en réseau et paramétrés.

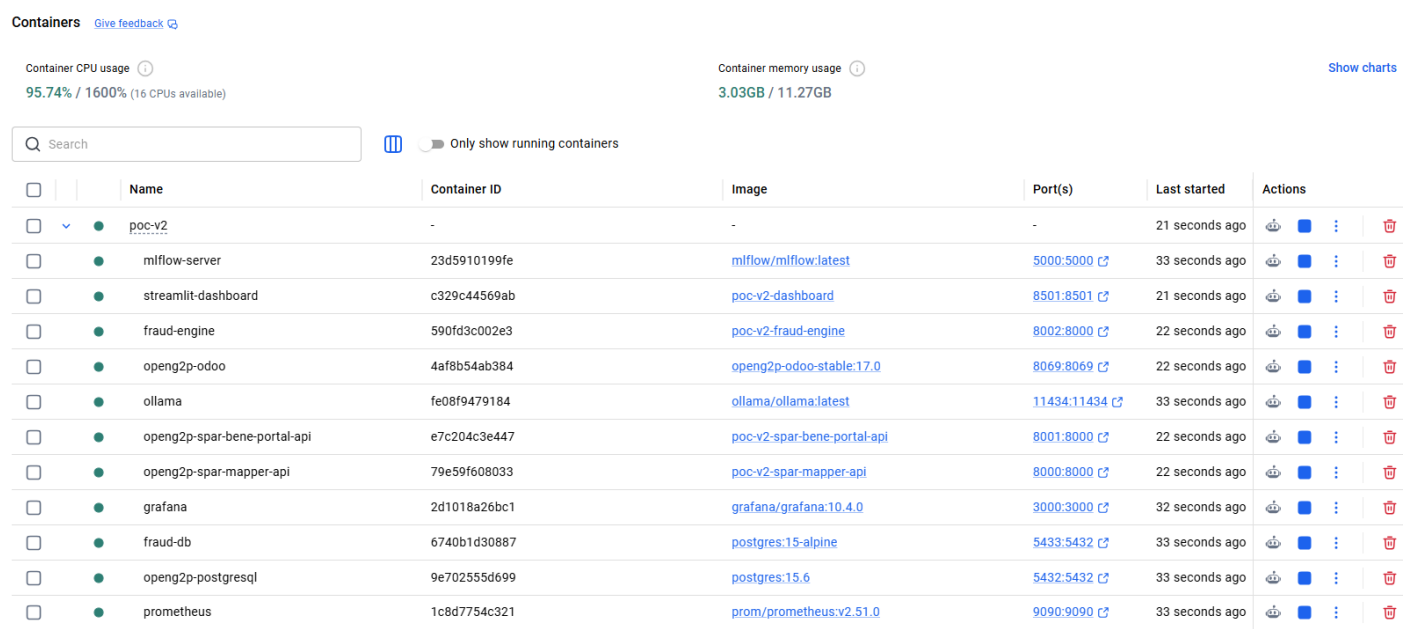


FIGURE 4.2 – Extrait de la configuration Docker Compose pour le moteur intelligent et la plateforme OpenG2P

Les Figures 4.3 et ??présentent la configuration complète du fichier `docker-compose.full.yml`, qui orchestre l'ensemble des services nécessaires au fonctionnement du moteur intelligent et de

la plateforme OpenG2P. Chaque service y est défini avec ses dépendances, ses variables d'environnement et ses volumes de données, afin d'assurer la persistance et l'isolation des données.

```
poc-v2 > docker-compose.full.yml > {} services
  > Run All Services | docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic applications (compose-spec.json) | ""
1  services:
2
3  # =====
4  # MLflow – Model Tracking & Visualization
5  # =====
6
7  > Run Service
8  mlflow:
9    image: ghcr.io/mlflow/mlflow:latest
10   container_name: mlflow-server
11   restart: unless-stopped
12   ports:
13     - "5000:5000"
14   command: mlflow server --host 0.0.0.0 --port 5000 --backend-store-uri sqlite:///mlflow/mlruns/mlflow.db
15   volumes:
16     - mlflow-data:/mlflow
17   environment:
18     MLFLOW_TRACKING_URI: http://localhost:5000
19   networks:
20     - openg2p-internal
21   healthcheck:
22     test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
23     interval: 10s
24     timeout: 5s
25     retries: 10
26
27 # =====
28 # OpenG2P – Services |
29 # =====
30
31 > Run Service
32 postgresql:
33   image: postgres:15.6
34   container_name: openg2p-postgresql
35   restart: unless-stopped
36   environment:
37     POSTGRES_DB: ${POSTGRES_DB:-openg2p}
38     POSTGRES_USER: ${POSTGRES_USER:-odoo}
39     POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set in .env}
40   volumes:
41     - db-data:/var/lib/postgresql/data
42   ports:
43     - "5432:5432"
44   healthcheck:
45     test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-odoo} -d ${POSTGRES_DB:-openg2p}"]
46     interval: 10s
47     timeout: 5s
48     retries: 10
49   networks:
50     - openg2p-internal
51
52 > Run Service
53 spar-mapper-api:
54   build:
55     context: ./openg2p-spar
56     dockerfile: Dockerfile.mapper
57   container_name: openg2p-spar-mapper-api
58   restart: unless-stopped
```

FIGURE 4.3 – Extrait de la configuration Docker Compose pour le moteur intelligent et la plateforme OpenG2P

```

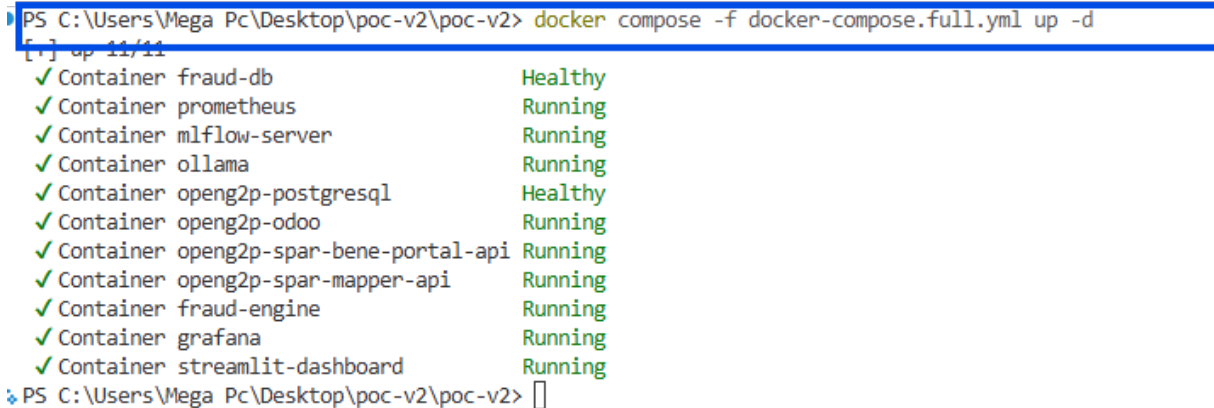
1  services:
    >Run Service
78  openg2p-odoo:
79      build:
80          context: .
81          dockerfile: Dockerfile.odoo
82      image: openg2p-odoo-stable:17.0
83      container_name: openg2p-odoo
84      restart: unless-stopped
85      depends_on:
86          postgresql:
87              condition: service_healthy
88      ports:
89          - "8069:8069"
90      environment:
91          ODOO_DATABASE_HOST: postgresql
92          ODOO_DATABASE_NAME: ${POSTGRES_DB:-openg2p}
93          ODOO_DATABASE_USER: ${POSTGRES_USER:-odoo}
94          ODOO_DATABASE_PASSWORD: ${POSTGRES_PASSWORD:?set in .env}
95          ODOO_LOAD_DEMO_DATA: "no"
96          ODOO_UPDATE: "no"
97          ODOO_ADDONS_PATH: "/opt/bitnami/odoo/addons,/opt/bitnami/odoo/extra-addons"
98          ODOO_ADMIN_PASSWORD: admin
99      volumes:
100          - odoo-data:/bitnami/odoo
101      networks:
102          - openg2p-internal
103  # =====
104  # FRAUD ENGINE – Nouveaux services
105  # =====
106
    >Run Service
107  fraud-db:
108      image: postgres:15-alpine
109      container_name: fraud-db
110      restart: unless-stopped
111      environment:
112          POSTGRES_DB: fraud_engine
113          POSTGRES_USER: fraud
114          POSTGRES_PASSWORD: fraud123
115      volumes:
116          - fraud_pgdata:/var/lib/postgresql/data
117      ports:
118          - "5433:5432"
119      networks:
120          - fraud-net
121      healthcheck:
122          test: ["CMD-SHELL", "pg_isready -U fraud -d fraud_engine"]
123          interval: 10s
124          timeout: 5s
125          retries: 3
    >Run Service
126  fraud-engine:
127      build:
128          context: ./fraud-detection-engine
129          dockerfile: Dockerfile
130      container_name: fraud-engine
131      restart: unless-stopped
132      ports:

```

FIGURE 4.4 – Extrait des conteneurs Docker Compose pour le moteur intelligent et la plateforme OpenG2P

Cette orchestration a été pensée pour minimiser la friction de déploiement : une seule commande suffit pour démarrer l'ensemble du système, comme le montre la Figure 4.5.

**docker compose -f docker-compose.full.yml up -d**



```
PS C:\Users\Mega Pc\Desktop\poc-v2\poc-v2> docker compose -f docker-compose.full.yml up -d
[+] up 11/11
✓ Container fraud-db           Healthy
✓ Container prometheus        Running
✓ Container mlflow-server      Running
✓ Container ollama             Running
✓ Container openg2p-postgresql Healthy
✓ Container openg2p-odoo       Running
✓ Container openg2p-spar-bene-portal-api Running
✓ Container openg2p-spar-mapper-api Running
✓ Container fraud-engine       Running
✓ Container grafana            Running
✓ Container streamlit-dashboard Running
❖ PS C:\Users\Mega Pc\Desktop\poc-v2\poc-v2>
```

FIGURE 4.5 – Démarrage de l'ensemble du système via Docker Compose

Cet environnement technique une fois posé, il reste à en dériver une organisation logicielle cohérente : comment le code est-il structuré pour refléter cette séparation entre plateforme sociale et moteur intelligent, et comment les composants du moteur communiquent-ils entre eux ? C'est l'objet de la section suivante.

## 4.2 Architecture logicielle

L'architecture logicielle traduit, au niveau du code, la séparation fonctionnelle établie au chapitre précédent entre la plateforme sociale OpenG2P et le moteur intelligent de détection de fraude. Cette section présente successivement l'organisation générale du projet, la structure interne du moteur intelligent, ses composants principaux, puis la manière dont ces composants interagissent pour produire une décision de fraude.

### 4.2.1 Organisation du projet

Le code est organisé en deux ensembles distincts : les modules OpenG2P, qui assurent le bon fonctionnement de la plateforme et l'interopérabilité entre Odoo et le moteur intelligent, et les modules du moteur intelligent, qui implémentent les couches de détection de fraude et d'intelligence artificielle. La Figure 4.6 donne une vision globale de cette organisation.

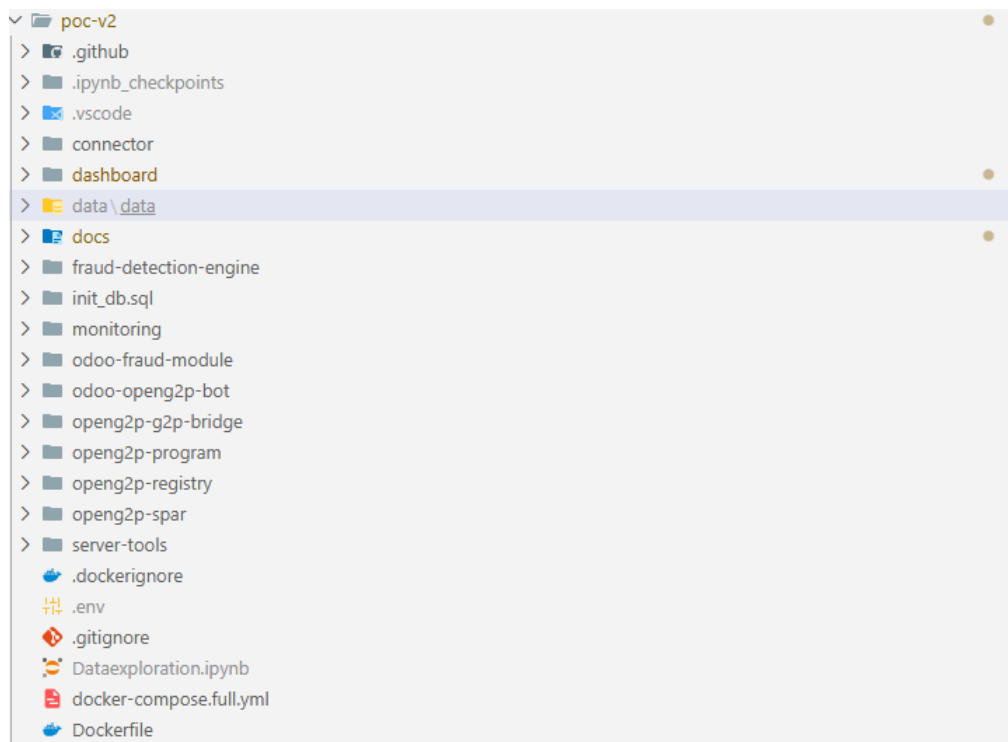


FIGURE 4.6 – Vision globale sur la structure du code source

### 4.2.2 Architecture du moteur intelligent

En zoomant sur le second ensemble, le moteur intelligent est lui-même structuré en couches, chacune responsable d'une préoccupation distincte. La Figure 4.7 détaille cette structure interne.

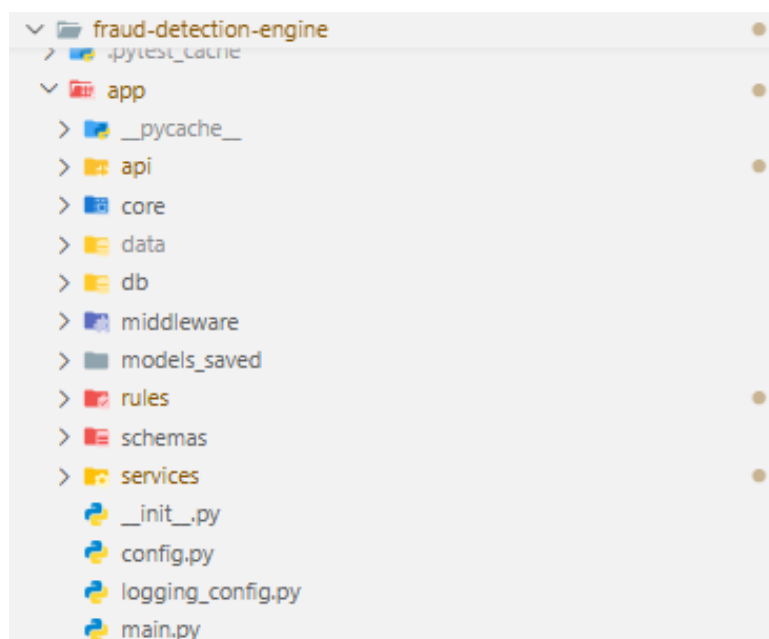


FIGURE 4.7 – Structure du code source du moteur intelligent de détection de fraude

- **Couche Présentation (API) : api** — Expose les services REST permettant la communication entre OpenG2P et le moteur antifraude.

- **Couche Métier (Business Layer) : services, rules** — Contient la logique métier, les règles de détection et le calcul du score de risque.
- **Couche Données (Data Layer) : db, data, schemas** — Gère l'accès aux données, leur stockage et leur validation.
- **Couche Intelligence Artificielle : models\_saved** — Regroupe les modèles de Machine Learning entraînés et leurs artefacts.
- **Couche Technique (Infrastructure) : core, middleware, config.py, logging\_config.py** — Assure la configuration, la sécurité, le middleware et les services techniques.
- **Point d'entrée de l'application : main.py** — Initialise et lance le moteur de détection de fraude.

Cette architecture en couches est conçue pour être modulaire et extensible : chaque composant peut être remplacé ou amélioré indépendamment, et de nouvelles sources de données ou modèles peuvent être intégrés sans perturber le reste du système. Reste à savoir ce que chacun de ces modules fait concrètement, et comment ils s'articulent pour transformer un profil de bénéficiaire en décision de fraude.

#### 4.2.3 Composants principaux du moteur

Chaque module de la couche métier a une responsabilité unique et bien délimitée. La Figure 4.8 présente la structure de ces composants principaux, dont le rôle est détaillé ci-après.

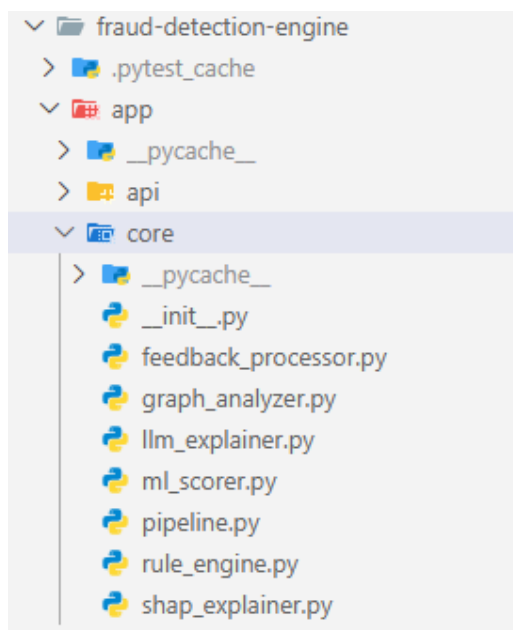


FIGURE 4.8 – La structure des composants principaux du moteur intelligent de détection de fraude

- **rule\_engine.py** : lit les règles depuis `fraud_rules.json`, évalue chaque condition de façon sécurisée (via l'AST Python), et retourne un `rule_score` entre 0 et 1.
- **ml\_scorer.py** : charge le modèle XGBoost depuis `models_saved/`, construit le vecteur de features et retourne une probabilité de fraude (`ml_score`).
- **graph\_analyzer.py** : construit un graphe à partir des numéros de téléphone et des comptes bancaires partagés entre bénéficiaires, puis applique PageRank pour identifier

les nœuds les plus risqués.

- **shap\_explainer.py** : calcule les valeurs SHAP sur la prédiction XGBoost et retourne les 5 features ayant le plus influencé le résultat.
- **llm\_explainer.py** : construit un prompt enrichi avec les scores, les flags et les facteurs SHAP, l'envoie à Mistral via Ollama, et retourne une explication en français naturel.
- **pipeline.py** : orchestre l'appel séquentiel des 4 moteurs, agrège les scores et retourne la réponse finale.

Cette décomposition répond à une question centrale : comment ces composants s'interconnectent-ils pour déterminer si un bénéficiaire est frauduleux ou non ? Le diagramme de séquence de la sous-section suivante répond précisément à cette question en traçant, étape par étape, le trajet complet d'une requête de scoring.

#### 4.2.4 Les interactions entre les composants du système

Une requête de scoring traverse le système selon un flux séquentiel bien défini, illustré à haut niveau par la Figure 4.9, avant d'être détaillé pas à pas.

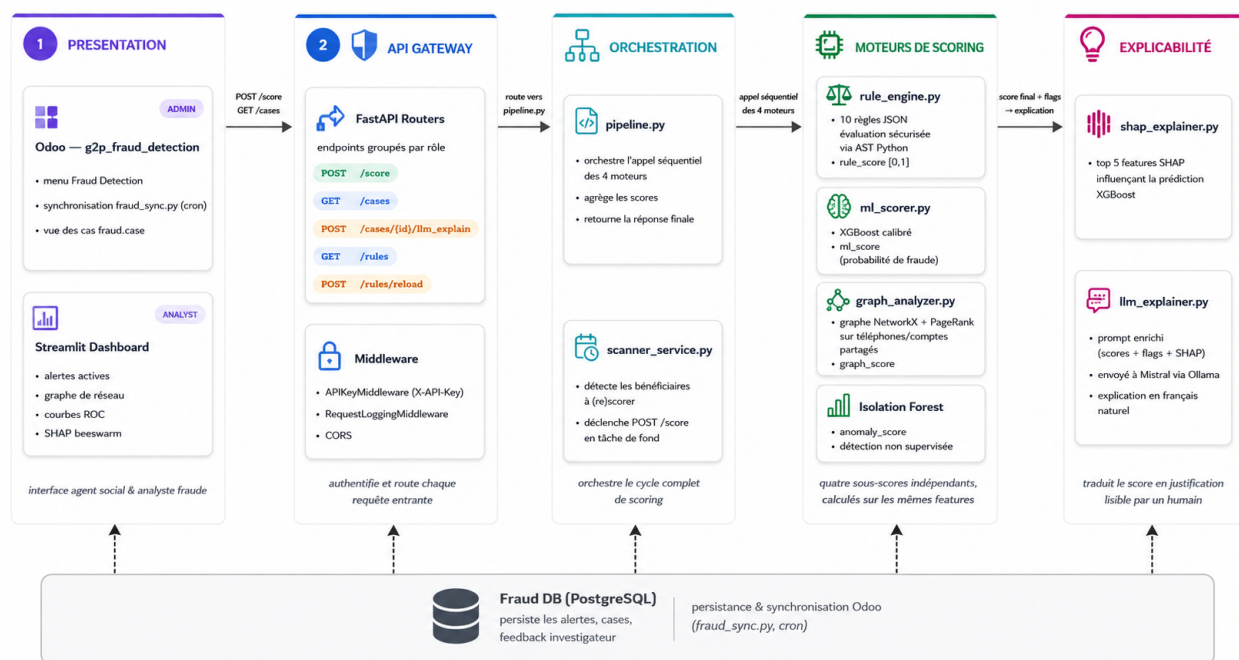


FIGURE 4.9 – Processus de scoring d'un bénéficiaire à travers les composants du moteur intelligent de détection de fraude

1. Le **scanner\_service** du moteur antifraude détecte un bénéficiaire à analyser (nouvel enregistrement ou re-scan périodique) et appelle **POST /score**.
2. FastAPI reçoit la requête et l'achemine vers **pipeline.py**.
3. Le pipeline appelle les 4 moteurs (**règles**, **XGBoost**, **Isolation Forest**, **PageRank**) qui s'exécutent sur les mêmes features.
4. Les 4 sous-scores sont agrégés en un score final (formule pondérée).
5. SHAP identifie les facteurs clés et Mistral rédige l'explication en langage naturel.

6. L'alerte est sauvegardée en base par le moteur antifraude.
7. Un cron Odoo (**`fraud_sync.py`**) interroge périodiquement **GET /cases** et crée les enregistrements **`fraud.case`** correspondants ; pour les cas **CRITICAL/HIGH**, il déclenche aussi automatiquement **POST /cases/{id}/llm\_explain** pour pré-générer l'explication IA.
8. L'analyste fraude consulte le cas dans **Odoo** (menu *Fraud Detection*), peut régénérer l'explication IA, consulter la heatmap Streamlit, ou faire évoluer le statut du dossier (investigation, confirmation, rejet).

## 4.3 Développement des fonctionnalités

### 4.3.1 Construction du dataset

Le moteur intelligent repose sur des algorithmes de Machine Learning, qui ne peuvent apprendre à distinguer un comportement frauduleux d'un comportement légitime qu'à partir d'exemples étiquetés. Or, **OpenG2P ne dispose d'aucune donnée de fraude** : la plateforme a été conçue pour gérer des programmes de protection sociale, pas pour documenter des cas de fraude confirmés. Construire un dataset d'entraînement exploitable est donc un préalable incontournable, et la première étape de la mise en œuvre du moteur intelligent.

Ce dataset ne peut cependant pas être improvisé : il doit être construit à partir des données réelles disponibles dans OpenG2P, structurées de façon à représenter fidèlement le profil et le comportement de chaque bénéficiaire. La Figure 4.10 illustre le processus retenu dans son ensemble, avant que chacune de ses étapes ne soit détaillée.

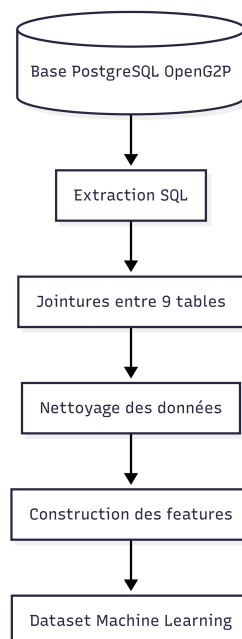


FIGURE 4.10 – Processus de construction du dataset de détection de fraude à partir des données OpenG2P

#### 4.3.1.1 Extraction SQL et jointures

La première difficulté rencontrée tient à la taille et à la complexité de la base OpenG2P elle-même. En tant qu'écosystème Odoo complet, elle compte plusieurs centaines de tables couvrant de nombreux modules sans rapport avec la détection de fraude (comptabilité, CRM, notifications, etc.), au sein desquelles il faut identifier le sous-ensemble réellement pertinent. La Figure 4.11 quantifie cet écart entre le schéma disponible et le schéma effectivement exploité : sur les 587 tables du schéma `public`, seules 7 sont mobilisées par l'extraction, et parmi celles-ci, seule une fraction des colonnes disponibles (par exemple 93 colonnes dans `res_partner`) alimente réellement les 25 features du modèle.

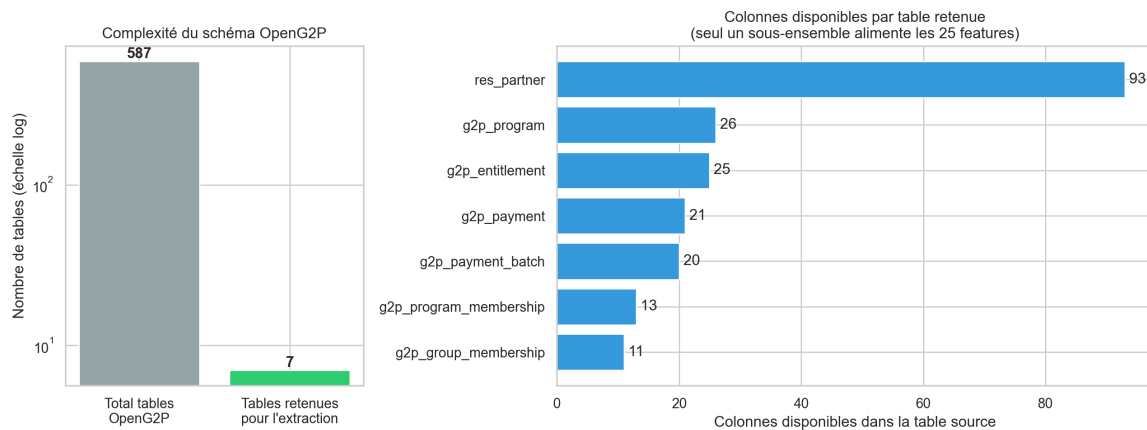


FIGURE 4.11 – Complexité du schéma OpenG2P (587 tables) et colonnes disponibles dans les 7 tables retenues pour l'extraction

Le module `feature_extractor.py` est responsable de l'extraction des informations depuis la base PostgreSQL d'OpenG2P. Afin de limiter les accès à la base de données, une requête SQL unique utilisant des Common Table Expressions (CTE) est exécutée. Cette requête réalise les jointures entre les tables retenues et agrège les informations relatives à chaque bénéficiaire en une seule ligne par profil — un prérequis technique indispensable avant de pouvoir formuler précisément ce que le moteur doit être capable de détecter.

#### 4.3.1.2 Prétraitement et scénarios de fraude simulés

Une fois la donnée rendue accessible, encore faut-il définir précisément l'objectif de détection : quels comportements, parmi la multitude de profils possibles, doivent être reconnus comme frauduleux ? Le prétraitement répond à cette question en simulant neuf scénarios de fraude reconnus dans la littérature sur la protection sociale, qui servent de référence pour étiqueter les exemples du dataset d'entraînement :



FIGURE 4.12 – Scénarios de fraude simulés

Un bruit gaussien réaliste est ajouté sur les features sensibles (revenus, score PMT, nombre de programmes) pour éviter une séparation parfaite entre les classes, ce qui rendrait le modèle peu généralisable en production. 8 % des fraudeurs reçoivent des features proches des légitimes (cas borderline), ce qui simule les cas difficiles à détecter — une exigence essentielle pour qu'un modèle entraîné sur ce jeu de données reste pertinent face à des fraudeurs réels qui cherchent activement à passer inaperçus.

Ces neuf scénarios une fois définis, la question suivante devient celle des moyens : quelles variables, parmi celles disponibles dans OpenG2P, permettent réellement de les détecter ?

#### 4.3.1.3 Sélection des sources de données

La solution retenue consiste à sélectionner, parmi l'ensemble des tables OpenG2P, celles qui contiennent un signal pertinent pour caractériser le profil socio-économique d'un bénéficiaire, sa participation aux programmes sociaux et son historique de paiements. L'analyse du

schéma relationnel de la base PostgreSQL a permis d'identifier les modules et tables listés au Tableau 4.2, qui constituent les principales sources utilisées pour construire les variables d'entrée du moteur de détection de fraude.

TABLEAU 4.2 – Principales sources de données exploitées pour la construction du dataset

| Module OpenG2P                     | Tables principales                 | Informations exploitées                                                                             |
|------------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Social Registry</b>             | res_partner                        | Informations d'identification du bénéficiaire (âge, sexe, téléphone, compte bancaire, revenu, etc.) |
| <b>Program Management</b>          | g2p_program<br>g2p_program_members | Programmes sociaux, inscriptions, statut d'adhésion et ancienneté du bénéficiaire.                  |
| <b>Entitlements</b>                | g2p_entitlement                    | Prestations attribuées aux bénéficiaires, droits ouverts et historique des allocations.             |
| <b>Paiements</b>                   | g2p_payment<br>g2p_payment_batch   | Historique des paiements, montants versés, statuts, cycles et fréquence des versements.             |
| <b>Registre des ménages</b>        | g2p_group<br>g2p_group_members     | Composition du ménage, liens familiaux et nombre de personnes à charge.                             |
| <b>Évaluation socio-économique</b> | PMT Assessment                     | Score PMT (Proxy Means Test) utilisé pour estimer le niveau de vulnérabilité des bénéficiaires.     |

Ces six sources couvrent l'identité du bénéficiaire, son inscription aux programmes, ses prestations, ses paiements, la composition de son ménage et son évaluation socio-économique — soit l'ensemble des dimensions nécessaires pour détecter aussi bien une fraude individuelle (revenus sous-déclarés) qu'une fraude organisée (réseau de comptes ou téléphones partagés). Ces variables sélectionnées, il reste à les transformer en un dataset structuré, directement exploitable par les algorithmes de Machine Learning.

#### 4.3.1.4 Construction des features et tableau des 25 variables

À l'issue du prétraitement, chaque profil de bénéficiaire est traduit en un vecteur de 25 features numériques, résumées par catégorie au Tableau 4.3. Ces variables couvrent la démographie, la composition du ménage, la participation aux programmes, le score PMT, l'historique de paiements, les signaux réseau et des indicateurs dérivés (flags).

TABLEAU 4.3 – Les 25 features extraites pour le modèle ML

| Catégorie          | Features                                                  | Signification                             |
|--------------------|-----------------------------------------------------------|-------------------------------------------|
| <b>Démographie</b> | age, income, income_per_person                            | Âge, revenu total et revenu par personne  |
| <b>Ménage</b>      | household_size, nb_children, nb_elderly, dependency_ratio | Composition et vulnérabilité du ménage    |
| <b>Ménage</b>      | has_disabled, single_head                                 | Présence d’handicap, chef de ménage seul  |
| <b>Programmes</b>  | nb_programs, nb_active_programs, avg_enrollment_days      | Inscriptions actives et ancienneté        |
| <b>Score PMT</b>   | pmt_score, pmt_score_min                                  | Score de ciblage proxy-means test         |
| <b>Paielements</b> | payment_count, payment_gap_ratio, payment_success_rate    | Volume et taux d’échec des paiements      |
| <b>Paielements</b> | amount_variance, cycle_count                              | Variabilité des montants reçus            |
| <b>Réseau</b>      | shared_phone_count, shared_account_count, network_risk    | Partage de ressources entre bénéficiaires |
| <b>Groupe</b>      | group_membership_count                                    | Appartenance à des groupes ménagers       |
| <b>Flags</b>       | high_amount_flag, income_program_inconsistent             | Signaux dérivés de règles heuristiques    |

Au-delà de ce regroupement par catégorie métier, un audit technique du schéma réel du fichier généré confirme la cohérence des types de données : les 25 features sont toutes numériques (entiers, flottants ou booléens), sans valeur manquante, ce qui les rend directement exploitables par les modèles de Machine Learning sans étape d’encodage supplémentaire. Ces variables une fois construites, elles doivent encore être assemblées en un dataset tabulaire cohérent, avant de pouvoir être généré à grande échelle et injecté dans OpenG2P.

#### 4.3.1.5 Structure du dataset final

À l’issue des différentes étapes d’extraction, de nettoyage et de construction des features, les informations relatives à chaque bénéficiaire sont regroupées dans un dataset tabulaire destiné à l’entraînement et à l’évaluation des modèles de détection de fraude.

Le dataset obtenu est organisé sous forme tabulaire, où chaque ligne représente un bénéficiaire et chaque colonne une information descriptive ou une feature calculée. Il comporte 36 colonnes, dont 25 features utilisées par les modèles de Machine Learning, les autres étant dédiées aux identifiants et aux informations de supervision. La Figure 4.13 présente la structure complète de ce dataset.

## Dataset OpenG2P — colonnes et types (36 colonnes, dont 25 features ML)

| Colonne                      | Type    | Valeurs non-nulles |
|------------------------------|---------|--------------------|
| age                          | int64   | 5,000              |
| income                       | float64 | 5,000              |
| income_per_person            | float64 | 5,000              |
| household_size               | int64   | 5,000              |
| nb_children                  | int64   | 5,000              |
| nb_elderly                   | int64   | 5,000              |
| dependency_ratio             | float64 | 5,000              |
| has_disabled                 | bool    | 5,000              |
| single_head                  | bool    | 5,000              |
| nb_programs                  | float64 | 5,000              |
| nb_active_programs           | int64   | 5,000              |
| pmt_score                    | float64 | 5,000              |
| pmt_score_min                | float64 | 5,000              |
| avg_enrollment_days          | int64   | 5,000              |
| payment_count                | int64   | 5,000              |
| payment_gap_ratio            | float64 | 5,000              |
| payment_success_rate         | float64 | 5,000              |
| amount_variance              | float64 | 5,000              |
| cycle_count                  | int64   | 5,000              |
| shared_phone_count           | float64 | 5,000              |
| shared_account_count         | float64 | 5,000              |
| network_risk                 | float64 | 5,000              |
| group_membership_count       | int64   | 5,000              |
| high_amount_flag             | int64   | 5,000              |
| income_program_inconsistency | int64   | 5,000              |
| is_fraud                     | int64   | 5,000              |
| synthetic_label              | int64   | 5,000              |
| partner_idx                  | int64   | 5,000              |
| partner_id                   | int64   | 5,000              |
| scenario                     | object  | 5,000              |
| registration_age_days        | int64   | 5,000              |
| avg_entitlement_amount       | float64 | 5,000              |
| total_issued                 | float64 | 5,000              |
| total_paid                   | float64 | 5,000              |
| network_risk_score           | float64 | 5,000              |
| elderly_head                 | bool    | 5,000              |

FIGURE 4.13 – Colonnes et types du dataset OpenG2P généré (36 colonnes, dont les 25 features ML)

Ce dataset structuré n'a de valeur que s'il peut être produit à l'échelle et alimenter réellement la plateforme : c'est l'objet de la dernière étape de cette construction.

#### 4.3.1.6 Génération du dataset synthétique et injection PostgreSQL

Une fois les features construites, le script `generate_dataset.py` permet de produire automatiquement un grand nombre de profils synthétiques reproduisant les caractéristiques observées dans les données OpenG2P. Ces données peuvent être injectées directement dans la base PostgreSQL grâce à l'option `-inject`, afin d'alimenter l'environnement de test avec un jeu de données cohérent et immédiatement exploitable.

```
1 docker exec fraud-engine python ml/scripts/generate_dataset.py \  
2     --n 10000 \  
3     --fraud-rate 0.12 \  
4     --inject
```

Listing 4.1 – Génération et injection du dataset

Lors de l'injection, le script respecte automatiquement les contraintes d'intégrité référentielle d'OpenG2P en suivant l'ordre de création des différentes entités (programmes, cycles, bénéficiaires, inscriptions, comptes bancaires, allocations puis paiements). Cette approche garantit l'obtention d'une base de données synthétique mais conforme au modèle relationnel de la plateforme.

Le dataset ainsi généré constitue le point d'entrée du moteur intelligent. Les 25 features extraites servent désormais de variables d'entrée communes aux différents mécanismes de détection de fraude présentés dans les sections suivantes : moteur de règles, modèles de Machine Learning, analyse de réseau et moteur d'explicabilité. Afin d'évaluer la robustesse des modèles supervisés dans un contexte transactionnel complémentaire, un second jeu de données, *PaySim*, est également utilisé lors de la phase d'entraînement et de validation.

#### 4.3.1.7 Un second banc d'essai : le dataset PaySim

*PaySim* est un jeu de données public (Kaggle, dérivé de journaux de transactions mobile money) simulant environ 470 000 transactions bancaires, dont un sous-ensemble de 172 473 lignes de type `TRANSFER` et `CASH_OUT` est retenu pour cette étude — les deux types d'opération réellement exposés à la fraude dans le simulateur. Contrairement au dataset OpenG2P, où chaque ligne représente un **bénéficiaire**, chaque ligne de PaySim représente une **transaction** horodatée : montant transféré, soldes du compte source et du compte destination avant et après l'opération, type d'opération, et libellé de fraude. Cette différence de grain — profil de bénéficiaire contre événement transactionnel isolé — signifie que les deux jeux de données ne partagent ni schéma ni identifiant commun, et ne peuvent donc pas être fusionnés en un seul dataset d'entraînement.

Le taux de fraude réel de PaySim est d'environ 0.3 %, très inférieur au 14 % du dataset OpenG2P généré. Un premier obstacle technique a dû être traité avant tout entraînement : le simulateur PaySim fabrique les transactions frauduleuses en mettant le solde du compte émetteur à zéro sans réellement soustraire le montant transféré, ce qui rend la colonne `balance_anomaly` (et les soldes bruts qui en dérivent) corrélée à près de 100 % avec le label de

fraude. Un modèle entraîné sur ces colonnes obtient un score parfait en apparence, sans avoir appris de signal réellement exploitable. Ces six colonnes ont donc été retirées, ne conservant que les 11 features suivantes : `tx_step`, `tx_amount`, `full_drain`, `round_amount`, `dest_was_empty`, `dest_is_merchant`, `log_amount`, `amount_to_balance_ratio`, `hour_of_day`, `is_night` et `tx_type_trans`.

La Figure 4.14 présente trois angles de lecture du dataset ainsi nettoyé. Le premier panneau montre que le montant de la transaction, seul, ne sépare pas nettement les transactions frauduleuses des transactions légitimes — la distribution des deux classes se chevauche largement. Le deuxième panneau révèle en revanche un signal temporel marqué : le taux de fraude grimpe fortement entre 2h et 8h du matin, avant de retomber à un niveau quasi nul le reste de la journée, ce qui confirme l'intérêt de la feature `is_night` pour la détection. Le troisième panneau compare la performance du meilleur modèle entraîné sur PaySim à celle du meilleur modèle entraîné sur OpenG2P, sur leurs jeux de test respectifs.

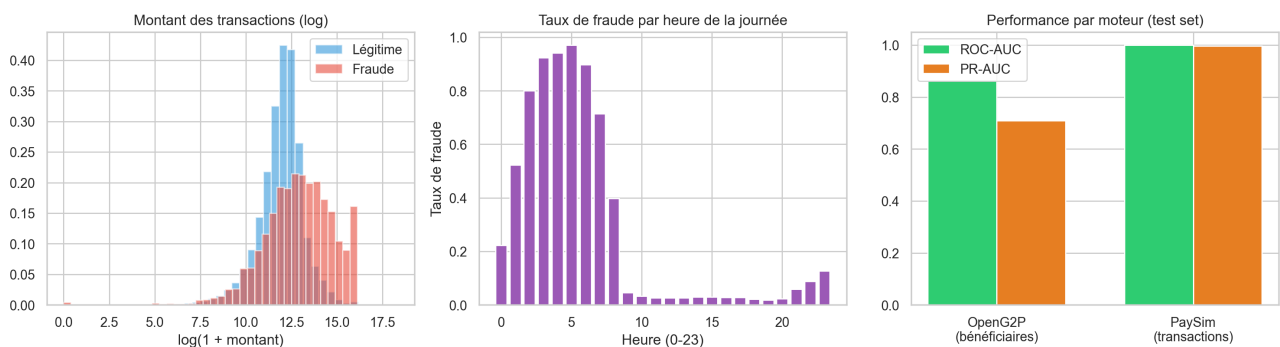


FIGURE 4.14 – Vue d'ensemble du dataset PaySim : distribution des montants par classe, taux de fraude par heure, et comparaison de performance avec le moteur OpenG2P

Le modèle XGBoost calibré entraîné sur PaySim atteint un ROC-AUC de 1.000 et un PR-AUC de 0.996 sur un jeu de test dont le taux de fraude réel (0.82 %) a été restauré après un split chronologique — une performance nettement supérieure à celle obtenue sur OpenG2P (ROC-AUC 0.863, PR-AUC 0.710). Cet écart ne signifie pas que le second problème est mieux résolu que le premier : il reflète plutôt une différence de nature entre les deux tâches. Une transaction bancaire frauduleuse laisse une empreinte numérique très caractéristique (vidage intégral du compte, montant non arrondi, horaire nocturne), alors qu'un profil de bénéficiaire frauduleux se distingue par des écarts plus subtils et retournant sur plusieurs dimensions (revenu, réseau, historique de paiement).

Comment ces résultats peuvent-ils alors profiter au moteur intelligent, si les deux jeux de données ne peuvent être fusionnés ? PaySim est utilisé comme un **second banc d'essai** pour les mêmes familles d'algorithmes (XGBoost, Isolation Forest), sur un contexte de fraude différent de celui d'OpenG2P. Cette validation croisée entre deux domaines apporte trois bénéfices concrets au moteur, sans toucher au score hybride actuel :

- **Détection de fuites de données** : la découverte de la fuite via `balance_anomaly` sur PaySim a permis d'auditer plus rigoureusement le pipeline de features d'OpenG2P à la recherche de fuites analogues (une feature calculée après le fait déclencheur de la fraude, par exemple), renforçant la confiance dans le caractère généralisable des 25 features retenues.

- **Choix de métriques adaptées au déséquilibre** : le déséquilibre extrême de PaySim (0.3 % de fraude réelle) a imposé l’usage systématique du PR-AUC et des indicateurs de type *precision@k* plutôt que du seul ROC-AUC — une pratique désormais appliquée également à l’évaluation du modèle OpenG2P.
- **Preuve de robustesse de la stack technique** : entraîner avec succès XGBoost et Isolation Forest sur un volume et un contexte différents (172 473 transactions contre 5 000 bénéficiaires) confirme que le pipeline d’entraînement du moteur n’est pas surajusté aux particularités du dataset OpenG2P généré.

Une intégration future du signal transactionnel au score hybride pondéré (voir la sous-section 4.3.5) nécessiterait de disposer d’un historique de paiement réel lié aux bénéficiaires — par exemple les tables `g2p_payment` d’OpenG2P enrichies de features analogues à celles de PaySim (montant, horaire, ratio au solde) — et non le dataset PaySim tel quel, qui reste un jeu de données public sans lien avec les bénéficiaires du système. Cette piste est présentée comme perspective d’évolution au dernier chapitre de ce rapport.

### 4.3.2 Moteur de détection de fraude

Le moteur de règles permet de capter des patterns de fraude connus sans avoir besoin d’un modèle ML — une première ligne de détection, déterministe et immédiatement interprétable, qui complète les modèles statistiques développés dans la suite du chapitre. Chaque règle est définie dans `fraud_rules.json` sous la forme d’une condition, d’un poids et d’un niveau de sévérité.

Les 10 règles couvrent les scénarios suivants : partage de compte bancaire (R001, poids 0.35), partage de téléphone (R002, poids 0.30), trop de programmes (R003, poids 0.30), score PMT suspect (R004, poids 0.20), écart de paiements (R005, poids 0.30), ménage fantôme (R006, poids 0.25), combinaison téléphone + compte (R007, poids 0.40), risque réseau élevé (R008, poids 0.30), sous-déclaration de revenus (R009, poids 0.20), et manipulation + réseau (R010, poids 0.35).

Les conditions sont évaluées de façon sécurisée via l’AST Python (`ast.NodeVisitor`), ce qui évite tout risque d’injection de code — un choix d’implémentation déterminant puisque les règles peuvent, à terme, être modifiées par un administrateur métier sans revalidation du code source.

Le `rule_score` est la somme des poids des règles déclenchées, plafonnée à 1.0. Une liste de flags (ex. : `SHARED_ACCOUNT`, `FRAUD_NETWORK_COMBO`) accompagne le score pour expliquer les déclenchements.

Si le moteur de règles capture efficacement les patterns de fraude déjà répertoriés, il reste par nature limité aux scénarios anticipés lors de sa conception. La détection de patterns plus subtils ou non explicitement codifiés nécessite un second moteur, statistique cette fois : le pipeline de Machine Learning.

### 4.3.3 Conception du moteur de détection de fraude par Machine Learning

Le pipeline ML entraîne trois modèles supervisés et un modèle non-supervisé, tous comparés via MLflow, afin de sélectionner objectivement le modèle offrant le meilleur compromis entre précision et rappel sur la détection de fraude.

**4.3.3.0.1 Rééquilibrage des classes avec SMOTE.** Le dataset est déséquilibré (88 % légitimes / 12 % fraudes) — un déséquilibre qui, sans traitement, biaiserait tout modèle supervisé vers la classe majoritaire. SMOTE (*Synthetic Minority Over-sampling Technique*) génère des exemples synthétiques de la classe minoritaire pour équilibrer les données d'entraînement :

```

1 from imblearn.over_sampling import SMOTE
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X, y, test_size=0.2, stratify=y, random_state=42
5 )
6
7 smote = SMOTE(sampling_strategy=0.5, random_state=42)
8 X_train_bal, y_train_bal = smote.fit_resample(X_train, y_train)

```

Listing 4.2 – Rééquilibrage avec SMOTE

**4.3.3.0.2 Entraînement XGBoost.** XGBoost est le modèle retenu pour sa capacité à capturer des interactions non-linéaires entre features.

**4.3.3.0.3 Isolation Forest pour la détection d'anomalies.** En complément du modèle supervisé, l'Isolation Forest détecte les bénéficiaires dont le comportement est statistiquement atypique, même sans label de fraude disponible — un filet de sécurité pour les fraudes émergentes qu'aucun exemple d'entraînement ne représente encore.

Le Tableau 4.4 compare les performances des trois modèles supervisés évalués sur le jeu de test.

TABLEAU 4.4 – Comparaison des modèles supervisés

| Modèle                          | AUC-ROC      | F1 (fraude)  | Précision    | Rappel       |
|---------------------------------|--------------|--------------|--------------|--------------|
| <b>XGBoost calibré (retenu)</b> | <b>0.858</b> | <b>0.630</b> | <b>0.895</b> | <b>0.486</b> |
| Random Forest                   | 0.841        | 0.608        | 0.871        | 0.470        |
| Régression Logistique           | 0.794        | 0.571        | 0.812        | 0.440        |

XGBoost est retenu pour son AUC la plus élevée et sa précision de 89.5 %, ce qui signifie que 9 alertes sur 10 générées sont de vraies fraudes — une propriété essentielle pour maintenir la confiance des agents sociaux dans le système.

Le moteur ML détecte efficacement les anomalies au niveau individuel, mais certaines fraudes ne se révèlent qu'au niveau collectif — lorsque plusieurs bénéficiaires partagent des ressources entre eux. C'est ce que l'analyse de réseau vient compléter.

#### 4.3.4 Analyse de réseau

Les fraudes organisées, comme le partage de compte bancaire entre plusieurs bénéficiaires fictifs, ne sont pas toujours visibles au niveau individuel. L’analyse de réseau les révèle en construisant un graphe où les nœuds sont les bénéficiaires et les arêtes représentent des ressources partagées.

Les comptes bancaires partagés ont un poids double (0.8 contre 0.5) car ils représentent un signal de fraude plus fort : partager un numéro de téléphone peut être légitime en contexte rural, mais partager un compte bancaire est très rarement justifiable. L’algorithme PageRank est ensuite appliqué sur ce graphe pour identifier les nœuds les plus centraux, donc les plus susceptibles d’appartenir à un réseau frauduleux organisé.

À ce stade, le système dispose de trois signaux indépendants — règles, ML, réseau — chacun capturant une facette différente du risque de fraude. Encore faut-il les combiner en une décision unique et exploitable : c’est le rôle de la fusion des scores.

#### 4.3.5 Fusion des scores

Les quatre moteurs produisent chacun un sous-score normalisé dans l’intervalle  $[0, 1]$ . Le score final est une combinaison pondérée :

$$\text{score\_final} = 0,70 \times s_{\text{ML}} + 0,20 \times s_{\text{règles}} + 0,10 \times s_{\text{graphe}} \quad (4.1)$$

Le poids dominant accordé au modèle ML (70 %) reflète sa capacité à capturer des interactions complexes entre features. Les règles métier (20 %) couvrent les cas évidents et déterministes. Le graphe (10 %) enrichit la détection des fraudes organisées sans dominer la décision. Ce score est ensuite converti en niveau de risque, sur une échelle à quatre paliers permettant de prioriser l’action des agents sociaux.

Un score de risque, aussi bien calibré soit-il, reste cependant une boîte noire pour l’agent social qui doit décider d’une action. Les deux couches suivantes — SHAP puis LLM Mistral — répondent à cette exigence d’explicabilité.

#### 4.3.6 Explicabilité SHAP

Un score seul n’est pas exploitable par un agent social : encore faut-il pouvoir justifier pourquoi un bénéficiaire donné a été signalé. Le moteur intelligent génère deux niveaux d’explication, dont le premier repose sur SHAP.

**4.3.6.0.1 SHAP** (*SHapley Additive exPlanations*) décompose la prédiction XGBoost en contributions individuelles de chaque feature, permettant d’identifier les cinq variables ayant le plus influencé le score attribué à un bénéficiaire donné.

Cette décomposition technique, bien qu’indispensable pour un data scientist ou un auditeur, demeure difficile à interpréter directement par un agent social non spécialiste. La dernière couche du pipeline traduit donc ces facteurs SHAP en une explication rédigée en langage naturel.

### 4.3.7 Explication LLM

**4.3.7.0.1 LLM Mistral** reçoit les facteurs SHAP, le score final et les flags déclenchés, et génère une explication lisible. Le RAG ChromaDB enrichit le contexte avec des cas de fraude similaires issus de la base de connaissance, ce qui permet à l'explication générée de s'appuyer sur des précédents comparables plutôt que sur les seules variables numériques.

**Exemple de réponse générée :**

*« Ce bénéficiaire présente plusieurs signaux d'alerte : son taux d'échec de paiement est anormalement élevé (68 %), son compte bancaire est partagé avec 3 autres personnes, et il est inscrit à 4 programmes simultanément. Ces éléments combinés suggèrent une fraude coordonnée. Une revue manuelle est recommandée avant tout versement. »*

Avec cette explication en langage naturel, le pipeline de scoring est complet : de l'extraction SQL jusqu'à la restitution d'une décision justifiée. Reste à situer ce pipeline dans son environnement d'exploitation réel — comment il est exposé via FastAPI, synchronisé avec Odoo, restitué dans un dashboard, et déployé en production. C'est l'objet de la section suivante.

## 4.4 Intégration dans OpenG2P

Le moteur intelligent, aussi complet soit-il, n'a de valeur opérationnelle que s'il s'intègre naturellement dans le quotidien des agents sociaux qui utilisent OpenG2P. Cette section décrit les quatre mécanismes qui assurent cette intégration : l'exposition du pipeline via FastAPI, la synchronisation automatique avec Odoo, la restitution visuelle dans le dashboard Streamlit, et le déploiement d'ensemble via Docker.

### 4.4.1 Exposition via FastAPI

FastAPI expose l'ensemble du pipeline décrit à la section précédente sous forme de services REST, ce qui permet à n'importe quel client — Odoo, Streamlit, ou un appel direct — de déclencher un scoring ou de consulter un cas sans connaître les détails d'implémentation du pipeline interne.

### 4.4.2 Synchronisation Odoo

Un module Odoo personnalisé (`g2p_fraud_detection`) synchronise périodiquement les cas de fraude détectés par le moteur intelligent et les affiche dans un menu dédié (*Fraud Detection*) sans quitter l'interface OpenG2P.

Un webhook (`POST /v1/webhook/beneficiary-saved`) permet également de déclencher un scoring en tâche de fond dès qu'un bénéficiaire est créé ou mis à jour dans OpenG2P, sans action manuelle de l'agent. Cette synchronisation garantit que la donnée reste fraîche côté Odoo sans imposer à l'agent social un aller-retour vers un outil distinct.

### 4.4.3 Dashboard Streamlit

Le dashboard Streamlit (`dashboard/app.py`) offre une vue consolidée des alertes en temps réel, complémentaire à l'intégration Odoo pour les usages d'analyse et de monitoring. Il est accessible à l'adresse `http://localhost:8501` et comporte quatre onglets :

- **Alertes actives** : liste des bénéficiaires CRITICAL et HIGH avec filtre par programme, niveau de risque et date.
- **Graphe de réseau** : visualisation interactive (PyVis) du réseau de fraude, avec les communautés détectées colorées.
- **Performance ML** : courbes ROC, matrice de confusion et évolution des métriques au fil des versions MLflow.
- **Analyse SHAP** : graphique *beeswarm* montrant l'impact de chaque feature sur l'ensemble des prédictions.

FIGURE 4.15 – Dashboard Streamlit : vue des alertes actives avec niveaux de risque

FIGURE 4.16 – Graphique SHAP beeswarm : importance des features sur l'ensemble des prédictions

### 4.4.4 Déploiement Docker

L'ensemble de ces composants — moteur intelligent, base de données, Odoo, dashboard, MLflow et Ollama — est déployé et orchestré via la configuration Docker Compose présentée en début de chapitre (Tableau 4.1, Figures 4.2 et 4.5), ce qui garantit que l'intégration décrite dans cette section repose sur un socle de déploiement unique, reproductible et versionné.

## 4.5 Conclusion partielle

Ce chapitre a présenté la mise en œuvre complète du moteur intelligent, depuis la génération des données d'entraînement jusqu'au déploiement de l'interface utilisateur. Toutes les fonctionnalités spécifiées au chapitre précédent ont été réalisées et intégrées dans un système cohérent et déployable via une seule commande Docker Compose.

### Récapitulatif des fonctionnalités développées :

- Génération d'un dataset synthétique de 10 000 bénéficiaires (12 % de fraudes, 9 scénarios) avec injection PostgreSQL
- Extraction de 25 features SQL multi-tables depuis OpenG2P avec gestion défensive des valeurs manquantes
- Moteur de règles métier : 10 règles JSON, évaluation sécurisée AST, `rule_score` avec liste de flags
- Modèle XGBoost calibré (AUC 0.858, précision 89.5 %) avec SMOTE, suivi MLflow et versioning des artefacts

- Isolation Forest pour la détection d'anomalies non-supervisée
- Graphe NetworkX + PageRank pour l'identification des réseaux frauduleux
- Score hybride pondéré (70 % ML + 20 % règles + 10 % graphe) avec 4 niveaux de risque
- Explicabilité SHAP (top 5 features) + LLM Mistral (explication en français naturel via RAG)
- Module Odoo `g2p_fraud_detection` avec synchronisation automatique et webhook de scoring
- Dashboard Streamlit : alertes, graphe de réseau, courbes ROC, graphique SHAP

Le chapitre suivant évalue la performance de ce système dans des conditions proches de la production : métriques ML approfondies, tests d'intégration avec OpenG2P, analyse des cas limites et mesure des temps de réponse.

# Webographie

---

- [1] Banque Mondiale. (2023). *World Development Indicators*. <https://data.worldbank.org/indicator>
- [2] OpenG2P Community. (2023). *OpenG2P Technical Documentation and Deployment Guide*. <https://openg2p.dev/docs/>
- [3] Banque Mondiale. (2020). *The Sourcebook of Social Protection Programmes*. Chapter 8 : Social Protection Finance and Risk. Washington, DC.